

Shelly / Zendure Multi-Device Controller

Technische Dokumentation

Zendure Multi-Device Controller - Gesamtdokumentation

Steuerskript zur lokalen, **cloud-unabhängigen** Regelung der Lade-/Entladeleistung von **einem oder mehreren** Zendure-SolarFlow-Systemen. Das Script unterstützt alle Zendure-Geräte mit zenSDK-Schnittstelle (SF800 und neuer).

(Animiertes Demo-GIF in der Online-Version verfügbar: siehe Projekt-Repository.)

Das Script ist als **Alternative** zu Home Assistant oder ähnlichem gedacht und läuft direkt als Shelly-Script auf einem Shelly Gen2/3/4-Gerät – ohne zusätzliche Infrastruktur (Server, Cloud, App) außer dem Shelly selbst und den Zendure-Geräten im lokalen Netz.

Inhaltsverzeichnis

- [Zendure Multi-Device Controller – Gesamtdokumentation](#)
 - [Inhaltsverzeichnis](#)
 - [1\) Zweck](#)
 - [2\) Möglichkeiten und Flexibilität](#)
 - [3\) Voraussetzungen](#)
 - [4\) Konfiguration](#)
 - * [4a\) Plausibilitätsprüfungen \(CONFIG-Validierung beim Start\)](#)
 - * [4b\) Mehrere Solarflow-Geräte einpflegen](#)
 - * [4c\) Testmodus: dryRun](#)
 - [5\) Smartmeter Anbindung - Die drei Grid-Quellen im Überblick](#)
 - * [Vorzeichen bei "http_json" prüfen](#)
 - * [Eigene/weitere Smart Meter einbinden "http_json"](#)
 - [6\) Regelverhalten](#)
 - * [6a\) Entladen](#)
 - * [6b\) Laden \(vom Netz\)](#)
 - * [6c\) Sanfter Ausgleich \(Rebalance\)](#)
 - [7\) Lastaufteilung / Wie wirken concentrateBelow und spreadAbove](#)
 - * [7a\) Mindestlast ist richtungsabhängig](#)
 - * [7b\) Empfehlung für Ersteinstellung](#)
 - [8\) Detailfunktionalität](#)
 - * [8.1\) Ablauf eines Regelzyklus](#)
 - * [8.2\) Dämpfung des kombinierten Signals](#)
 - * [8.3\) Watchdog / Timer](#)
 - * [8.4\) Fehlerzähler & Benachrichtigung](#)
 - * [8.5\) Signal-Benachrichtigung \(CallMeBot\) – inkl. WhatsApp](#)
 - * [8.6\) SoC-Grenzwert-Synchronisierung beim Start](#)
 - * [8.7\) Startprotokoll \(Banner\)](#)
 - * [8.8\) Zeitlich gekoppelte Hysterese \(spread → single\)](#)

- * 8.9) Start/Stop-Hysterese (Laden & Entladen)
- * 8.10) Logausgaben
- 9) Ablauf der Regelung (Diagramm)
 - * 9.1) Cooldown-Timer für Richtungswechsel (Beispiel)
- 10) Hinweis: Performance, Grenzen und Status-Prüfung des Shelly
 - * 10.1) Debug
- 11) Bekannte Einschränkung
- 12) getestete Beispielkonfiguration
- 13) Live-Override der Regelparameter per KVS (z. B. Home Assistant)

In Kürze

- **Install & Run:** Pro Gerät nur die IP-Adresse in CONFIG.devices[] eintragen – und die Grundeinstellungen pro Gerät, wie SoC-Grenzen und Leistungswerte, alles andere ist sinnvoll voreingestellt. [Script herunterladen](#)
 - Konfigurierbar ist aber z.B.:
 - Sollwert
 - Hysterese
 - Dämpfung
 - Taktung
 - maximale Leistung je Gerät
 - Minimale und Maximale Soc-Einstellung mit Übergabe an die Solarflows
 - Start/Stop-Schwellen für Laden und Entladen (global, siehe Kapitel 8.9)
 - Cooldown-Timer gegen zu häufige Richtungswechsel (siehe Kapitel 9.1)
 - AC-Laden/Überschussladen aktivierbar (reverse: false)
 - Entladen pro Gerät sperrbar, um z. B. das Laden zu priorisieren (dischargeAllowed: false)
 - Ladeleistung einstellbar
 - Benachrichtigungsfunktion aktivierbar (Signal, Whatsapp...über Callmebot)
-

1) Zweck

Das Script regelt die Lade-/Entladeleistung von einem oder mehreren Zendure-Speichern automatisch so, dass der Netzbezug/-export des Haushalts möglichst auf einen gewünschten Zielwert (in der Regel 0 W) ausgeglichen wird (Nulldurchgangsregelung / Eigenverbrauchsoptimierung). Es liest den aktuellen Netzwert und den Zustand aller konfigurierten Zendure-Geräte, berechnet daraus einen gemeinsamen Leistungs-Sollwert und verteilt diesen anschließend nach konfigurierbaren Regeln auf die einzelnen Geräte. Optional kann die/können die Batterie(n) bei Bedarf auch **aktiv aus dem Netz geladen** werden (bidirektionale Regelung, pro Gerät einzeln zuschaltbar).

Kernprinzipien:

- **Lokal & cloud-unabhängig:** Die Netzleistung wird lokal im Netzwerk gemessen, die Kommunikation mit den Zendure-Geräten erfolgt per lokaler HTTP-API. Kein Umweg über die Zendure-Cloud – das Script funktioniert auch bei Internetausfall.
- **Flexibler Einsatzort:** Über die Betriebsart gridSource kann das Script entweder direkt auf einem Shelly Pro 3EM laufen ("local"), auf einem beliebigen anderen Shelly-Gerät laufen und die Netzleistung per RPC von einem Pro 3EM abrufen ("remote"), oder die Netzleistung generisch von **jedem** Smart Meter mit einer JSON-über-HTTP-Schnittstelle beziehen ("http_json", z. B. Zendure Smart Meter 3CT). Details siehe Kapitel 5.
- **Laden (optional, pro Gerät):** Neben dem normalen Export/Entladen kann jedes Gerät einzeln über reverse: true bei Bedarf auch gezielt aus dem Netz geladen werden – sin-

nvoll z. B. wenn ein weiterer Wechselrichter im Hausnetz einspeist und ein rechnerisches Defizit entsteht.

- **Reaktive Regelung:** Alle paar Sekunden wird die aktuelle Netzleistung gemessen sowie der Zustand aller Geräte abgefragt, und die kombinierte Ziel-Leistung wird gedämpft nachgeführt und auf die Geräte verteilt.
 - **Ausfallerkennung:** Fehlerhafte Messwerte, nicht erreichbare Geräte oder fehlgeschlagene Schreibvorgänge werden – global und pro Gerät – erkannt, gezählt und optional per Signal- oder WhatsApp-Nachricht gemeldet.
-

2) Möglichkeiten und Flexibilität

Was die Regelung kann:

- **Beliebig viele Zendure-Geräte** gleichzeitig steuern (CONFIG.devices ist ein Array – ein Gerät hinzufügen heißt: einen weiteren Eintrag ergänzen, der Rest des Scripts passt sich automatisch an).
 - **Gewichtete Verteilung nach SOC:** Geräte mit höherem Ladestand tragen beim Entladen mehr bei, Geräte mit mehr freiem Platz tragen beim Laden mehr bei – automatisch, ohne feste Prioritäten.
 - **Einer oder Alle:** Bei kleinem Bedarf wird nur ein einzelnes Gerät angesprochen (**Konzentrationsmodus**, schont die übrigen Akkus, weniger Schreibzugriffe), bei größerem Bedarf wird automatisch auf alle verfügbaren Geräte verteilt (**Verteilmodus** nach Prinzip "Wasserfüllung").
 - **Kapazitäts-Deckelung mit Nachverteilung:** Jedes Gerät ist einzeln auf sein maxOutput/maxInputPower begrenzt; was ein gedeckeltes Gerät nicht abnehmen kann, wird automatisch auf die übrigen Geräte umverteilt.
 - **Cutoffs:** Geräte unterhalb minSoc werden vom Entladen ausgeschlossen, Geräte bei/über maxSoc werden vom Laden ausgeschlossen.,
 - **Laden vom Netz ist optional und pro Gerät einzeln freischaltbar** (reverse: true/false); Geräte mit reverse:false werden nie zum Laden herangezogen. Zur Laufzeit per KVS umschaltbar (Kapitel 13).
 - **Entladen ist optional und pro Gerät einzeln sperrbar** (dischargeAllowed: true/false); Geräte mit false werden nie zum Entladen herangezogen, können aber weiterhin (bei reverse:true) laden – z. B. um ein Gerät gezielt fürs Laden zu reservieren. Zur Laufzeit per KVS umschaltbar (Kapitel 13).
 - **SoC Balancing, Ausgleich** zwischen mehreren Geräten im Konzentrationsmodus: Ein dauerhafter, ausreichend großer SOC-Vorsprung eines anderen Geräts führt zu einem Wechsel des aktiven Geräts.
 - **Drei Quellen für den Netzmesswert:** lokal auf einem Shelly Pro 3EM, per RPC von einem entfernten Shelly Pro 3EM, oder generisch von jedem Gerät mit einer JSON-über-HTTP-Schnittstelle (z. B. Zendure Smart Meter 3CT).
 - **Dry-Run-Modus pro Gerät:** Ein Geräteeintrag kann auf "nur lesen und mitrechnen, nie schreiben" gestellt werden – nützlich, um die Verteil-Logik mit echten Live-Daten zu testen, ohne eine reale Schreib-Aktion auszulösen.
 - **Automatische Fehlererkennung und -erholung** pro Gerät und global (Verbindung, Antwortformat, Seriennummer, Schreibvorgang, Netzmesswert, Watchdog), inklusive optionaler Signal-/WhatsApp-Benachrichtigung nach mehreren aufeinanderfolgenden Fehlschlägen.
 - **Debug-Logging** ein-/ausschaltbar über CONFIG.debug, ohne Code löschen/wieder einfügen zu müssen.
-

3) Voraussetzungen

- **Shelly-Gen2/3/4-Geräten** mit Script-Engine und ausreichendem Script-Speicher
 - Shelly 3EMPro - erfolgreich getestet
 - Shelly 3EM-63W Gen3 - erfolgreich getestet
 - Shelly 1PM Gen3 - erfolgreich getestet
 - Shelly 1PM Gen4 - erfolgreich getestet
 - Shelly PlugS Gen3 - **nicht** erfolgreich getestet (out of memory)
- Einen **Smartmeter** für den aktuellen Netz-Leistungswert (siehe **Kapitel 5** für Details je Betriebsart).
 - Shelly 3EMPro
 - Zendure D0
 - Zendure CT
 - alle REST API fähigen Smartmeter, Tasmota....
- Ein oder mehrere **Zendure-Geräte** im selben lokalen Netz, jeweils mit fester/bekannter IP-Adresse. Die lokale HTTP-API muss aktiv sein (/properties/report, /properties/write).
=> **siehe Kapitel 12 Test-Setup**
- **HEMS-Kopplung entfernen:** Alle Zendure-Geräte müssen in der Zendure-App **aus dem HEMS (Home Energy Management System)** entfernt werden, da sonst die App/Cloud parallel eigene Steuerbefehle sendet und mit diesem Script kollidiert.
- **Netzwerkverbindung** zwischen dem Shelly, auf dem das Script läuft, und allen Zendure-Geräten (gleiches LAN/WLAN, keine Firewall-Blockade auf den genutzten Ports).
- **Für Signal-/WhatsApp-Benachrichtigungen (optional):**
 - Ein bei [CallMeBot](#) registrierter Signal- **oder** WhatsApp-Zugang mit API-Key (siehe Kapitel 8.5).
 - Internetzugang des Shelly zum Versenden der Benachrichtigung (die Kernregelung selbst benötigt dafür **kein** Internet – nur der optionale Alarmversand).

4) Konfiguration

[Script herunterladen](#)

i INFO: [Wie das mit den Scripte im Shelly funktioniert](#)

1) Vorweg zu konfigurieren im Script ist zunächst nur der Geräteblock.

```
{
ip: "192.168.68.XXX",           // Die IP-Adresse Eures Hubs
label: "SF2400AC",            // ein Name, frei wählbar

minSoc: 15,                   // die erlaubte Entladegrenze (%)
maxOutput: 2400,              // MaxPower Eures Hubs (W)

reverse: true,                // true, wenn Euer Hub auch AC-Laden kann und darf, sonst false
maxSoc: 100,                  // die obere Ladegrenze (%)
maxInputPower: 2400,          // maximale Ladeleistung (W)

dischargeAllowed: true,       // true = darf entladen/exportieren, false = nur laden (Priorität)

dryRun: false                 // false ist Standard, true ist für Simulationszwecke
},
```

i Kein minOutput mehr pro Gerät: Die frühere Mindestleistung ist ab Version 1.5.9/2.0.0 durch zwei **globale** Start/Stop-Schwellen ersetzt (dischargeStartupPower/dischargeStopPower, siehe Kapitel 8.9) – ein Gerät läuft dadurch entweder mit sinnvoller Leistung oder gar nicht, nie dauerhaft mit winzigen, ineffizienten Werten.

Habt Ihr mehrere Hubs am Laufen? Block mit geschweiften Klammern und abschließendem Komma kopieren und anpassen, => Siehe **Kapitel 4b**

2) **Zweiter** wichtiger Schritt: Die Konfiguration Eures Smartmeters => **siehe Kapitel 5**

3) **Dritter** wichtiger Schritt: Konfigurieren der Schwellwerte für den Kooperationsmodus mehrerer Geräte => **Kapitel 7b**

★ Damit läuft das Script schon. Alle anderen Parameter kann man später einstellen und verstehen.

Alle Einstellungen befinden sich im CONFIG-Objekt am Anfang des Scripts. Die folgende Tabelle bündelt sämtliche Parameter.

Bereich	Feld	Standardwert	Beschreibung
Geräte (CONFIG.devices[], ein Eintrag pro Zendure-Gerät)	ip	- (muss gesetzt werden)	IP-Adresse des Zendure-Geräts im lokalen Netz. Entspricht dem zendure-Parameter der Einzelgerät-Version, jetzt aber pro Geräteeintrag.
	label	-	Freier Name für Logs und Benachrichtigungen (z. B. "SF800", "SF2400"). Hat keine Auswirkung auf die Regelung selbst.
	minSoc	15-18 % (Beispiel)	Unterhalb dieses Ladezustands wird dieses Gerät von der Entlade-Verteilung ausgeschlossen (Gewicht 0, siehe Kapitel 6a). Pendant zum globalen minSoc der Einzelgerät-Version, jetzt individuell pro Gerät einstellbar.

Bereich	Feld	Standardwert	Beschreibung
	maxOutput	800 W	Maximale Entlade-/Exportleistung dieses Geräts – Kappungsgrenze in Konzentrations- und Verteilmodus.
	dischargeAllowed	true	Darf dieses Gerät entladen/exportieren? Geräte mit false erhalten in der Entlade-Gewichtung immer 0 und werden nie herangezogen (z. B. um das Laden dieses Geräts zu priorisieren). Laden (reverse) bleibt davon unberührt. Zur Laufzeit per KVS umschaltbar, siehe Kapitel 13.
	reverse	false	Darf dieses Gerät gezielt aus dem Netz geladen werden? Geräte mit false erhalten in der Lade-Gewichtung immer 0 und werden nie herangezogen. Zur Laufzeit per KVS umschaltbar, siehe Kapitel 13.
	maxSoc	100 %	Obergrenze für das Laden aus dem Netz dieses Geräts: ab diesem SOC wird der Ladezweig für dieses Gerät gesperrt (Gewicht 0). Pendant zum globalen maxSoc der Einzelgerät-Version, jetzt pro Gerät. Wird zusätzlich beim Start als socSet an die Geräte-Firmware synchronisiert (siehe Kapitel 8.6).

Bereich	Feld	Standardwert	Beschreibung
Netzquelle (global)	maxInputPower	z. B. 1200–2400 W	Maximale Ladeleistung vom Netz für dieses Gerät.
	dryRun	false	true = Gerät wird normal gelesen und fließt vollständig in Gewichtung/Verteilung ein, aber es wird nie tatsächlich beschrieben . Siehe Kapitel 4c.
	gridSource	"local"	Betriebsart der Netzleistungsmessung: "local" = Script läuft direkt auf dem Shelly Pro 3EM; "remote" = Script läuft auf einem beliebigen anderen Shelly-Gerät und liest den Pro 3EM per RPC über das Netzwerk aus; "http_json" = Script liest die Netzleistung generisch per HTTP GET von einem beliebigen JSON-Smart-Meter. Details siehe Kapitel 5.
	gridSourceIp	-	IP-Adresse des Shelly Pro 3EM. Nur relevant, wenn gridSource = "remote".
	gridSourceEmId	0	EM-Kanal-ID am Pro 3EM (in der Regel 0). relevant, wenn gridSource = "local" or "remote".
(http_json)	gridSourceUrl	"http://<IP-of-your-meter>/properties/repo-JSON"	Vollständige URL des generischen JSON-Smart-Meters. Nur relevant, wenn gridSource = "http_json".

Bereich	Feld	Standardwert	Beschreibung
(http_json)	gridSourceField	"total_power"	Name des JSON-Felds mit der gesamten Netzleistung in Watt. Nur relevant, wenn gridSource = "http_json".
(http_json)	gridSourceInvert	false	true, falls das Vorzeichen von gridSourceField umgekehrt zur Erwartung des Scripts ist. Siehe Kapitel 5 zur Vorzeichenprüfung.
Regelverhalten (global)	interval	4000 ms	Zykluszeit der Regelschleife. Bei mehreren Geräten steigt die tatsächliche Zyklusdauer, da alle Geräte-Anfragen sequenziell laufen (siehe Kapitel 10) – bei Bedarf großzügiger wählen.
	setpoint	0 W	Ziel-Netzleistung, auf die geregelt wird (0 = Nullregelung; negativ = leichter Export; positiv = leichter Import erlaubt). Gilt für die kombinierte Leistung aller Geräte zusammen.
	hysteresis	10 W	Mindeständerung pro Gerät gegenüber dem zuletzt an dieses Gerät geschriebenen Wert, bevor ein neuer Schreibvorgang an dieses Gerät ausgelöst wird. Wird für jedes Gerät individuell geprüft.

Bereich	Feld	Standardwert	Beschreibung
	dampingFactor	0.6	Dämpfungs-/Verstärkungsfaktor ($0 < x \leq 1$) des kombinierten Sollwerts, bevor dieser auf die Geräte verteilt wird. Der Wert springt nicht direkt zum berechneten Ziel, sondern nähert sich ihm pro Zyklus nur um diesen Anteil der Restdifferenz an. 1.0 = keine Dämpfung. Details und Unterschied zur Einzelgerät-Version siehe Kapitel 8.2.
Verteil-Schwellen	discharge.concentrationAbove	600 / 800 W	Schwellen für die Umschaltung zwischen einem Gerät (Konzentration) und allen Geräten (Verteilung) beim Entladen – siehe Kapitel 7.
	charge.concentrationAbove	600 / 800 W	Schwellen für die Umschaltung zwischen einem Gerät (Konzentration) und allen Geräten (Verteilung) beim Laden

Bereich	Feld	Standardwert	Beschreibung
	concentrateHoldMinutes	3min	Zeitlich gekoppelte Hysterese nur für den Wechsel von Verteil- auf Konzentrationsmodus (spread→single): Der Zielwert muss ununterbrochen (mit Einfrieren in der Totzone zwischen den Schwellen) für diese Zeit unter concentrateBelow bleiben, bevor gewechselt wird. Gilt gemeinsam für discharge und charge. Wird beim Start in Zyklen umgerechnet ($\text{CONCENTRATE_HOLD_CYCLES} = \text{concentrateHoldMinutes} \times 60000 / \text{interval}$, siehe Kapitel 8.8). Der umgekehrte Wechsel (single→spread) bleibt davon unberührt und weiterhin sofort.

Bereich	Feld	Standardwert	Beschreibung
SoC Ausgleich	rebalance.socMargin	10 Prozentpunkte	Nötiger Gewichts-Vorsprung eines anderen Geräts gegenüber dem aktuell aktiven Gerät im Konzentrationsmodus, damit ein Wechsel ausgelöst wird. Hinweis: Im aktuellen Script (zerooutput_multi.js) ist dies das <i>einzig</i> e rebalance-Feld – ein holdMinutes-Parameter existiert im CONFIG-Objekt nicht (mehr). Der Wechsel erfolgt sofort , sobald der Vorsprung erreicht ist, siehe Kapitel 6c und 8.5.
Netzladen (global)	reverseStartupPower	30 W	Mindest-Ladebedarf (kombiniert, systemweit), ab dem das Laden vom Netz überhaupt erst gestartet wird – Totzone gegen unnötiges Anspringen.
	reverseStopPower	10 W	Bedarfs-/Leistungsschwelle, unterhalb derer das Laden wieder gestoppt wird (muss $\leq$ reverseStartupPower sein – wird beim Start automatisch korrigiert).

Bereich	Feld	Standardwert	Beschreibung
Entladen (global)	dischargeStartupPower	35 W	Mindest-Entladebedarf (kombiniert, systemweit), ab dem das Entladen überhaupt erst gestartet wird – Pendant zu reverseStartupPower, aber für die Entladerichtung. Wirkt einmalig auf Aggregatebene in calculate(), nicht pro Geräteanteil. Ersetzt das frühere pro-Geräte minOutput. Details und Beispiel siehe Kapitel 8.9.
	dischargeStopPower	15 W	Leistungsschwelle pro Gerät/Geräteanteil , unterhalb derer dieses Gerät auf 0 W gesetzt wird (muss < dischargeStartupPower sein – wird beim Start automatisch korrigiert). Gilt sowohl im Konzentrations- als auch im Verteilmodus, für jedes Gerät einzeln.

Bereich	Feld	Standardwert	Beschreibung
AC-Richtungswechsel (global)	directionChangeCooldown	8000 ms	Mindestzeit nach einem echten acMode-Wechsel (Laden↔Entladen) an einem Gerät, bevor dieses Gerät die Richtung erneut wechseln darf. Schützt reine AC-ladefähige Solarflows ohne eigene smarte Umschaltrampe vor zu häufigem Hin-und-Her, z. B. bei sekundlich springenden Smartmeter-Werten. Während der Sperre wird die alte Richtung mit 0 W gehalten. 0 = Feature deaktiviert. Details und Beispiel siehe Kapitel 9.1.
	httpTimeout	5 s	Timeout pro einzelner HTTP-Anfrage . Sollte spürbar kürzer sein als watchdog, damit jeder einzelne Call sicher abgeschlossen (erfolgreich oder fehlgeschlagen) ist, bevor der Watchdog überhaupt greifen könnte.
Timer&DEBUG	watchdog	10000 ms	Maximale Zeit für einen vollständigen Zyklus (Netz lesen + alle Geräte lesen + Verteilung berechnen + alle Schreibvorgänge), bevor er zwangsweise zurückgesetzt wird. Siehe Kapitel 8.3.

Bereich	Feld	Standardwert	Beschreibung
KVS (Live-Override, siehe Kapitel 13)	errorThreshold	5	Anzahl aufeinanderfolgender Fehler desselben Typs (global oder pro Gerät), bevor eine Signal-/WhatsApp-Benachrichtigung ausgelöst wird. Siehe Kapitel 8.4.
	kvsEnabled	true	Hauptschalter für das gesamte KVS-Live-Override-Feature. false = kein KVS.GetMany-Aufruf mehr pro Zyklus, kein Seeding beim Start – alle Live-Override-Parameter (setpoint, hysteresis, dampingFactor, dischargeAllowed, reverse) bleiben dann fest auf den CONFIG-Werten, ein KVS.Set von außen hat keine Wirkung mehr.
	kvsForceReseed	false	Nur relevant bei kvsEnabled: true. true = bei jedem Skriptstart werden alle zdmc_*-KVS-Keys mit den aktuellen CONFIG-Werten überschrieben, bereits live per KVS gesetzte Werte gehen dabei verloren. △ Siehe Warnhinweis in Kapitel 13.

Bereich	Feld	Standardwert	Beschreibung
	debug	false	Ausführliches Logging der Schreib-Anfragen (exakte URL/Body, vollständige Fehlerantwort) sowie Zyklus-Timing-Meldungen. Für Diagnosezwecke temporär aktivieren, im Normalbetrieb false lassen.
Signal / WhatsApp (CallMeBot)	signal.enabled	false	Aktiviert/deaktiviert Benachrichtigungen insgesamt.
	signal.typ	"SIGNAL"	"SIGNAL" oder "WHATSAPP" – wählt den CallMeBot-Endpunkt/Messenger aus. Details siehe Kapitel 8.5.
	signal.phone	"PHONE-STRING"	Telefonnummer bzw. Phone-ID für CallMeBot (z. B. +4917XXXXXXX).
	signal.apiKey	"YOUR_API_KEY"	API-Key von CallMeBot.

Minimal notwendige Anpassung vor dem ersten Start: Für **jedes** Gerät in CONFIG.devices muss mindestens ip gesetzt werden. Alle anderen Werte sind optional und mit sinnvollen Standardwerten vorbelegt.

4a) Plausibilitätsprüfungen (CONFIG-Validierung beim Start)

Um Konfigurationsfehler zu vermeiden, werden folgende Validierungen & ggf. Korrekturen durchgeführt um Parameter in sinnvolle Wertebereich zu bringen. Die Regel in folgender Tabelle beschreibt den gültigen Wertebereich. Wird dagegen verstoßen, werden die Parameter auf die jeweiligen Grenzen gesetzt.

⚠ Die Validierung findet nur beim Start statt.

Parameter	Regel	Korrektur bei Verstoß
interval	≥ 2500 ms	auf 2500 gesetzt
watchdog	$\geq 2.5 \times \text{interval}$	auf $2.5 \times \text{interval}$ gesetzt
dampingFactor	zwischen 0.1 und 1	auf 0.1 bzw. 1 gesetzt
concentrateBelow (discharge/charge)	≥ 35	auf 35 gesetzt
spreadAbove (discharge/charge)	≥ 50	auf 50 gesetzt

Parameter	Regel	Korrektur bei Verstoß
concentrateBelow vs. spreadAbove	concentrateBelow <= spreadAbove	spreadAbove = concentrateBelow + 15
reverseStartupPower vs. reverseStopPower	reverseStartupPower > reverseStopPower	reverseStartupPower = reverseStopPower + 10
dischargeStopPower	≥ 0	auf 0 gesetzt
dischargeStartupPower vs. dischargeStopPower	dischargeStartupPower > dischargeStopPower	dischargeStartupPower = dischargeStopPower + 10
directionChangeCooldown	≥ 0	auf 0 gesetzt
directionChangeCooldown vs. interval	directionChangeCooldown $\geq 1.5 \times \text{interval}$ (sofern nicht 0/deaktiviert)	directionChangeCooldown = $1.5 \times \text{interval}$
setpoint	$-50 < \text{setpoint} < 50$	$-50 = \text{setpoint} = 50$
hysteresis	$5 < \text{hysteresis} < 50$	$5 = \text{hysteresis} = 50$
socMargin	$3 < \text{socMargin} < 25$	$3 = \text{socMargin} = 25$

Kein eigenes Logging: Der Validierungsblock selbst gibt keine Korrektur-Meldungen aus – die jeweils gültigen Endwerte sind ohnehin im Start-Banner (Kapitel 8.7) sichtbar.

Warum $1.5 \times \text{interval}$ bei directionChangeCooldown: Ein Cooldown, der kleiner oder gleich interval ist, wäre spätestens beim nächsten Poll schon wieder abgelaufen (elapsed $\geq$ cooldown träfe sofort zu) und hätte dann de facto **keine** Wirkung, obwohl aktiv konfiguriert – eine stille Fehlkonfiguration. $1.5 \times$ ist die kleinste Marge, die noch deterministisch **mindestens einen vollen Zyklus** blockiert. Der Faktor ist bewusst kleiner als der $2.5 \times$ -Faktor bei watchdog: Letzterer deckt die gesamte Zyklus-**Verarbeitungszeit** ab (alle Geräte-Reads/-Writes können bei mehreren Geräten länger als ein interval dauern), während der Cooldown-Vergleich nur an der reinen Poll-**Taktung** hängt.

4b) Mehrere Solarflow-Geräte einpflegen

Ein weiteres Zendure-Gerät hinzuzufügen bedeutet **keine Code-Änderung**, sondern nur einen weiteren Eintrag im CONFIG.devices-Array:

1. In CONFIG.devices ein neues Objekt { ... }, ergänzen (Komma nach dem vorherigen Eintrag nicht vergessen).
2. Mindestens ip setzen, label sinnvoll benennen (taucht so in Logs und Benachrichtigungen auf).
3. minSoc, maxOutput passend zum jeweiligen Gerätetyp einstellen (z. B. 800 W als optimale Momentanleistung für ein SF2400AC/2400Pro, siehe Empfehlung in Kapitel 7b).
4. reverse, maxSoc, maxInputPower nur relevant, falls dieses Gerät auch aus dem Netz laden darf.
5. dryRun: false für reale Geräte setzen (true nur zu Testzwecken, siehe Kapitel 4c).
6. Optional: discharge/charge $\rightarrow$ concentrateBelow/spreadAbove neu berechnen, da diese Schwellen von der Geräteanzahl N abhängen (Faustformel in Kapitel 7b: spreadAbove = $N \times \text{optimalPowerGerät}$).
7. Bei vielen Geräten interval und watchdog ggf. großzügiger wählen, da alle Geräte-Anfragen sequenziell laufen und die Zyklusdauer linear mit der Geräteanzahl steigt (siehe Kapitel 10).
8. Script neu starten – der interne Zustand (aktives Gerät, Fehlerzähler, Glättung) beginnt dabei bei null (siehe Kapitel 10).

Die gesamte übrige Logik (Gewichtung, Verteilung, Fehlerbehandlung, Rebalancing) passt sich automatisch an die neue Geräteanzahl an.

4c) Testmodus: dryRun

Ein Geräteeintrag mit `dryRun: true` wird normal gelesen (SOC, Ist-Leistung) und **fließt vollständig in Gewichtung und Verteilung ein** – es wird aber **nicht tatsächlich beschrieben**. Statt eines Schreibvorgangs erscheint im Log nur `[DRYRUN] wurde schreiben: X W`.

Typischer Einsatz - Split-Logik mit nur einem echten Gerät testen:

- Einen zweiten (dritten, ...) Eintrag mit **derselben ip** wie das echte Gerät anlegen, aber `dryRun: true` und abweichenden Werten (`minSoc`, `maxOutput`, ...).
- `calculate()` dedupliziert die Ist-Leistung nach `ip` – ein reales Gerät fließt trotz mehrerer Einträge nur **einmal** in Summe Geraete ein. Ohne diese Absicherung würde dieselbe Leistung doppelt gezählt und eine Rückkopplungsschleife mit immer weiter steigendem Zielwert auslösen.
- Für einen sichtbaren Split zwischen echtem und Test-Eintrag müssen sich die Gewichte tatsächlich unterscheiden (z. B. `minSoc`), sonst gewinnt bei Gleichstand immer der erste Eintrag im Array.

Ladeverhalten (reverse) gefahrlos testen, auch mit nur einem realen, nicht-reverse-fähigen Gerät:

- Echtes Gerät: `reverse: false` (bleibt dadurch in `computeChargeWeights()` immer mit Gewicht 0 ausgeschlossen – unabhängig vom Testaufbau).
- Zusätzlicher `dryRun`-Eintrag (gleiche `ip`): `reverse: true` mit den zu testenden Werten (`maxSoc`, `maxInputPower`).
- Um ohne echten Netzbezug einen Lade-Zyklus zu erzwingen, `CONFIG.setpoint` testweise deutlich erhöhen (z. B. auf 200) – danach nicht vergessen, wieder auf den ursprünglichen Wert (i. d. R. 0) zurückzusetzen.

Woran man `dryRun`-Zeilen im Log erkennt: Anhang `[DRYRUN - wird nicht geschrieben]` an der Soll-Zeile des Geräts, gefolgt von `[DRYRUN] wurde schreiben: X W (Export) bzw. (Laden vom Netz)`.

5) Smartmeter Anbindung - Die drei Grid-Quellen im Überblick

Kurz gesagt: "local" ist am schnellsten und zuverlässigsten, läuft aber zwingend auf dem Pro 3EM selbst. "remote" und "http_json" geben dir Freiheit, das Script auf **jedem** Shelly laufen zu lassen — der Unterschied liegt nur darin, *welches* Messgerät im Netz abgefragt wird. Diese Betriebsarten gelten identisch für die Single- und die Multi-Device-Version; das Script wählt intern dieselbe Auswertungsfunktion (`handleGenericGridResponse()`).

	1) "local"	2) "remote"	3) "http_json"
Wo läuft das Script?	zwingend auf dem Shelly Pro 3EM selbst	auf jedem Shelly Gen2/3-Gerät	auf jedem Shelly Gen2/3-Gerät
Messquelle	<code>Shelly.getComponentSoc()</code> – interner Gerätespeicher	<code>HTTP GET auf "http://192.168.1.1/rpc/EM.GetStatus"</code> eines Shelly Pro 3EM	HTTP GET auf eine beliebige, frei konfigurierbare JSON-URL

	1) "local"	2) "remote"	3) "http_json"
Unterstützte Geräte	nur der Pro 3EM, auf dem das Script selbst läuft	ausschließlich Shelly Pro 3EM (Standard-Firmware, unverändert)	beliebige Smart Meter mit flachem JSON über HTTP GET, z. B. Zendure Smart Meter 3CT, aber auch zukünftige/andere Fabrikate
Relevante Config-Felder	gridSourceEmId	gridSourceIp, gridSourceEmId	gridSourceUrl, gridSourceField, gridSourceInvert
Feldname der Leistung	fest total_act_power (Shelly-intern)	fest total_act_power	frei konfigurierbar über gridSourceField (z. B. "total_power" beim 3CT)
Latenz	keine – synchroner, lokaler Zugriff, kein Netzwerk-Roundtrip	ein zusätzlicher HTTP-Call pro Zyklus	ein zusätzlicher HTTP-Call pro Zyklus
Netzwerk nötig?	nein	ja, zum Pro 3EM	ja, zum jeweiligen Meter
Besonderheit	schnellste & robusteste Variante, da kein HTTP-Fehlerfall möglich	Pro 3EM bleibt unangetastet mit Standard-Firmware; Script selbst kann auf schwächerer Hardware laufen	Vorzeichen vorher prüfen! Nicht jeder Hersteller definiert „positiv = Bezug vom Netz“ gleich – ggf. gridSourceInvert: true setzen

Vorzeichen bei "http_json" prüfen

Das Script erwartet durchgängig: **positiv = Import/Bezug vom Netz, negativ = Export ins Netz**. Bei "local" und "remote" ist das durch die Shelly-eigene total_act_power-Definition automatisch korrekt. Bei "http_json" hängt es vom jeweiligen Hersteller ab – das muss einmalig geprüft werden:

1. gridSourceInvert: false lassen und Script starten.
2. Einen größeren Verbraucher im Haus einschalten (mehr Netzbezug erwartet).
3. In der Shelly-Konsole die Zeile Grid: ... W beobachten.
4. Wird der Wert **positiver** → Vorzeichen passt, nichts weiter zu tun.
5. Wird der Wert **negativer** → gridSourceInvert: true setzen und erneut testen.

Eigene/weitere Smart Meter einbinden "http_json"

"http_json" ist bewusst generisch gehalten, damit sich künftig auch andere Fremd-Smart-Meter ohne Codeänderung anbinden lassen – es müssen nur drei Werte angepasst werden:

Generisch Konfiguration

```
gridSource: "http_json",
gridSourceUrl: "http://<IP-des-Messgeraets>/<passender-Pfad>",
gridSourceField: "<Feldname-mit-der-Gesamtleistung-in-Watt>",
gridSourceInvert: false, // ggf. nach Test auf true setzen
```

Voraussetzung ist lediglich, dass das Gerät auf einen einfachen HTTP GET ein **JSON-Objekt** zurückgibt (keine Authentifizierung) und darin ein Feld mit der **gesamten** Netzleistung in Watt enthalten ist. Das Feld selbst herausfinden geht am einfachsten, indem man die URL im Browser aufruft und die Antwort durchsucht.

Beispiel Zendure Smart Meter 3CT (flache JSON-Struktur, keine Verschachtelung):

Befehl im Browser absetzen: `http://<IP>/properties/report` liefert

```
→ {"timestamp":..., "deviceId": "...", "a_aprt_power": 0, "b_aprt_power": 16,
    "c_aprt_power": -14, "total_power": 2}
```

Hier ist `total_power` das gesuchte Feld.

```
gridSource: "http_json",
gridSourceUrl: "http://<IP-des-Geraets>/properties/report",
gridSourceField: "total_power",
gridSourceInvert: false // erstmal so lassen, dann pruefen (siehe unten)
```

Beispiel Tasmota (verschachtelte JSON-Struktur):

Befehl im Browser absetzen: `http://<IP>/cm?cmd=STATUS%2010` liefert

Achtung **BEISPIEL**

```
{"StatusSNS": {"Time": "2024-07-23T21:18:52", "DVS7420": {"energy": 20302.00, "power": 353.150, "...
```

Hier ist `power` das gesuchte Feld, aber in einer geschachtelten Struktur, die in `gridSourceField` mitzugeben ist... die Namen VOR den geschweiften Klammern sind WICHTIG Wir müssen die Felder VOR den öffnenden geschweiften Klammern heraussuchen und in der angegebenen Form in das Feld `gridSourceField` eintragen.

Wenn obiges also die Antwort ist, dann

```
gridSource: "http_json",
gridSourceUrl: "http://<IP-des-Geraets>/cm?cmd=STATUS%2010",
gridSourceField: ["StatusSNS", "DVS7420", "power"],
gridSourceInvert: false // erstmal so lassen
```

6) Regelverhalten

6a) Entladen

1. Nur Geräte mit `dischargeAllowed: true` (Standard) nehmen überhaupt teil; Geräte mit `false` bekommen automatisch Gewicht 0 und werden nie zum Entladen herangezogen – unabhängig vom SOC. Für jedes übrige, verfügbare Gerät wird ein **Gewicht** berechnet: $SOC - minSoc$. Ein Gerät bei oder unter `minSoc` bekommt ebenfalls Gewicht 0 und wird von der Verteilung automatisch ausgeschlossen.
2. Anhand des kombinierten Ziel-Werts wird entschieden, ob **ein einzelnes Gerät** (Konzentrationsmodus) oder **alle verfügbaren Geräte gemeinsam** (Verteilmodus) angesprochen werden – siehe Kapitel 7 für die genaue Schwellenlogik. Der Wechsel auf ein einzelnes Gerät ist dabei **zeitlich verzögert** (`concentrateHoldMinutes`), der umgekehrte Wechsel auf mehrere Geräte bleibt **sofort** (siehe Kapitel 8.8).
3. **Konzentrationsmodus:** Es wird das Gerät mit dem höchsten Gewicht gewählt ("sticky" – bleibt über mehrere Zyklen aktiv, kein Flackern). Reicht dessen `maxOutput` für das Ziel nicht aus, wechselt das Script **sofort** in den Verteilmodus, unabhängig von den konfigurierten Schwellen.

4. **Verteilmodus (Wasserfüllung):** Der Zielwert wird proportional zum Gewicht auf alle aktiven Geräte aufgeteilt. Erreicht ein Gerät dabei sein `maxOutput`, wird es auf diesen Wert gedeckelt und der verbleibende Rest auf die übrigen, noch nicht gedeckelten Geräte neu verteilt (iterativ, bis nichts mehr übrig ist oder niemand mehr Kapazität hat).
5. **Start/Stop-Hysterese (global):** Bevor überhaupt entladen wird, muss der kombinierte Bedarf die systemweite Schwelle `dischargeStartupPower` überschreiten (Anlaufschwelle gegen Flackern bei minimalem Bedarf) – **außer** es wird bereits entladen, dann genügt zum Weiterlaufen die niedrigere Schwelle `dischargeStopPower`. Diese Start-Prüfung wirkt **einmalig auf die Gesamtsumme**, bevor die Verteilung überhaupt beginnt. Zusätzlich wird **pro Gerät/Geräteanteil** geprüft: Fällt der einem Gerät zugewiesene Anteil unter `dischargeStopPower`, wird dieses Gerät auf 0 W gesetzt statt auf einen Mindestwert hochgezogen – ein Gerät läuft entweder mit sinnvoller Leistung oder gar nicht. Ausführliches Beispiel mit allen vier Start/Stop-Schwellen (Laden **und** Entladen) siehe Kapitel 8.9.
6. Reicht die addierte Kapazität aller verfügbaren Geräte nicht für das Ziel aus, liefert das Script einfach so viel wie möglich – der Rest bleibt ungedeckt, es gibt keine Fehlermeldung dafür.
7. **Nach der Verteilung:** Bevor ein berechneter Wert tatsächlich an ein Gerät geschrieben wird, prüft `applyOutputs()/writeDevice()` zusätzlich den `directionChangeCooldown` – wechselt die Richtung (`acMode`) dieses Geräts gegenüber dem zuletzt geschriebenen Wert, kann der Schreibvorgang kurzzeitig zurückgehalten werden. Details und Beispiel siehe Kapitel 9.1.

6b) Laden (vom Netz)

1. Nur Geräte mit `reverse: true` nehmen überhaupt teil; alle anderen bekommen automatisch Gewicht 0.
2. Das Gewicht ist gespiegelt zum Entladen: $\text{maxSoc} - \text{SOC}$. Ein Gerät bei/über `maxSoc` bekommt Gewicht 0.
3. Bevor überhaupt geladen wird, muss der Bedarf die systemweite Schwelle `reverseStartupPower` überschreiten (Anlaufschwelle gegen Flackern bei minimalem Überschuss). Läuft das Laden bereits, genügt zum Weiterlaufen die niedrigere Schwelle `reverseStopPower`.
4. Danach gilt exakt dieselbe Logik wie beim Entladen (Konzentration vs. Verteilung inkl. der zeitlich gekoppelten Hysterese aus `concentrateHoldMinutes`, Sticky-Auswahl, Wasserfüllung, Kapazitäts-Override), nur gespiegelt und gedeckelt auf `maxInputPower` statt `maxOutput`. Vergleichendes Beispiel beider Richtungen (Laden **und** Entladen) mit allen vier Start/Stop-Werten siehe Kapitel 8.9. Zum abschließenden Cooldown vor dem eigentlichen Schreibvorgang siehe Kapitel 9.1.

6c) Sanfter Ausgleich (Rebalance)

- Gilt nur im **Konzentrationsmodus** (ein Gerät aktiv) – im **Verteilmodus** nicht relevant.
- Aktives Gerät bleibt aktiv ("sticky"), keine Neubewertung nach reinem Gewichtsvergleich in jedem Zyklus.
- **Sofortiger Wechsel** (ohne Verzögerung): aktives Gerät wird ungültig – nicht erreichbar, oder Gewicht 0 durch `minSoc/maxSoc`.
- **Wechsel zu einem besseren Kandidaten**, sobald: $\text{Vorsprung} \geq \text{rebalance.socMargin}$ Prozentpunkte.
- Entladen (`state.discharge`) und Laden (`state.charge`) haben getrennte, unabhängige Zähler/Auswahl.

7) Lastaufteilung / Wie wirken concentrateBelow und spreadAbove

Steuern, ab wann zwischen “ein Gerät” (Single, Konzentration) und “alle Geräte” (Verteilung, spread) umgeschaltet wird – als **asymmetrische** Hysterese:

- single → spread: **sofort**, sobald das Ziel **über spreadAbove** steigt.
- spread → single: **nicht mehr sofort**. Das Ziel muss zusätzlich **ununterbrochen für concentrateHoldMinutes Minuten unter concentrateBelow** bleiben, bevor tatsächlich auf ein einzelnes Gerät gewechselt wird. Die genaue Zähl-/Einfrier-Logik dieser Haltezeit steht in Kapitel 8.8. Diese zeitliche Kopplung wurde ergänzt, weil eine reine Wert-Hysterese bei periodisch schwankenden Lasten knapp um die Schwellen herum (Wolken/Sonne-Wechsel, taktende Großverbraucher wie Fritteusen) sonst zu häufigem Modus-Flackern zwischen Konzentration und Verteilung führen kann.
- Zwischen den beiden Schwellen (concentrateBelow ... spreadAbove, die “Totzone”) bleibt der zuletzt aktive Modus unverändert; die Haltezeit-Zählung für den spread→single-Wechsel läuft in diesem Bereich **weder weiter noch wird sie zurückgesetzt** (eingefroren).
- concentrateBelow muss $\leq$ spreadAbove sein (wird beim Start automatisch korrigiert).
- **Ausnahme:** Reicht das gewählte Einzelgerät (maxOutput/maxInputPower) nicht für das Ziel, wird **sofort** in den Verteilmodus gewechselt – unabhängig von Schwellen und Haltezeit.

Die Schwellenwerte concentrateBelow/spreadAbove sind weiterhin getrennt für discharge/charge konfigurierbar; die Haltezeit concentrateHoldMinutes ist dagegen ein gemeinsamer, globaler Wert für beide Richtungen (siehe Kapitel 4).

7a) Mindestlast ist richtungsabhängig

Ein Gerät arbeitet allein bis zum Erreichen von (spreadAbove). Der Hausbedarf wird auf die Anzahl der Geräte N verteilt. Beim **Verharren nach fallendem Ziel** sinkt die Last bis auf concentrateBelow / N – dieser Wert entscheidet über das Verlassen des Modus, nicht spreadAbove.

Beispiel (3 Geräte, maxOutput 2400 W, gleiches Gewicht), concentrateBelow: 1440, spreadAbove: 2400:

Aufwärts (600 → 3000 W):

Ziel (W)	Modus	Gerät 1	Gerät 2	Gerät 3
600–2400	single	= Ziel	0	0
2500	spread	833	833	833
2600	spread	867	867	867
2700	spread	900	900	900
2800	spread	933	933	933
2900	spread	967	967	967
3000	spread	1000	1000	1000

Abwärts (3000 → 600 W):

Ziel (W)	Modus	Gerät 1	Gerät 2	Gerät 3
3000–2500	spread	Ziel/3	Ziel/3	Ziel/3
2400	spread	800	800	800
2300	spread	767	767	767
2000	spread	667	667	667

Ziel (W)	Modus	Gerät 1	Gerät 2	Gerät 3
1500	spread	500	500	500
1400	single	1400	0	0
1300-600	single	= Ziel	0	0

- Bei 2400 W liefern beide Richtungen unterschiedliche Ergebnisse – direkter Beleg der Hysterese.
- Tiefster Punkt: 500 W (bei Ziel 1500 W) $\approx \text{concentrateBelow} / N = 480 \text{ W}$ – nicht die vollen $\text{spreadAbove} / N = 800 \text{ W}$.

i Dieses Beispiel ist unabhängig von den Start/Stop-Schwellen ($\text{dischargeStartupPower}/\text{dischargeStopPower}$ Kapitel 8.9) – alle gezeigten Werte liegen weit über deren Standard-Bereich (35 W/15 W) und wären in der Praxis in jedem Fall aktiv.

7b) Empfehlung für Ersteinstellung

- N = Anzahl Geräte
- optimalPowerGerät = z.B. 2400 W für einen SF2400Pro
- spreadAbove kleiner oder gleich optimalPowerGerät
- $\text{concentrateBelow} \approx 0,6 \times \text{spreadAbove}$

Kompromiss zwischen Hysterese-Puffer und Mindestlast als Startpunkt.

Beispiel: SF2400Pro - $\text{spreadAbove} = 2400 - \text{concentrateBelow} \approx 2000$

Beispiel: SF800 - $\text{spreadAbove} = 800 - \text{concentrateBelow} \approx 600$

Beispiel: NICHT zm EMPFEHLEN - $\text{spreadAbove} = 0 - \text{concentrateBelow} \approx 0$

schaltet den Konzentrationsmodus aus, es laufen immer alle Geräte in jedem Lastmodus (ineffizient)

8) Detailfunktionalität

8.1) Ablauf eines Regelzyklus

1. **Timer** löst alle interval ms (Standard 4 s) $\text{update}()$ aus.
2. $\text{update}()$ prüft state.busy :
 - Läuft ein vorheriger Zyklus noch → Zyklus wird übersprungen ("Vorheriger Zyklus läuft noch").
 - Andernfalls: $\text{lock}()$ setzt $\text{state.busy} = \text{true}$, erhöht state.cycleId und startet einen Watchdog-Timer (watchdog ms , siehe 8.3).
3. $\text{readGridPower}()$ ermittelt den aktuellen Netz-Leistungswert – je nach CONFIG.gridSource lokal, per RPC (remote) oder generisch per HTTP-JSON (http_json), siehe Kapitel 5. Schlägt das fehl, wird ein Fehler vom Typ em gemeldet, $\text{unlock}()$ aufgerufen und der Zyklus abgebrochen – **ohne** dass überhaupt ein Gerät gelesen wird.
4. $\text{readAllDevices}()$ fragt **sequenziell** (ein Gerät nach dem anderen, nicht parallel) für jedes konfigurierte Gerät $\text{/properties/report}$ ab: SOC, aktuelle Ist-Leistung (zenPower , aus $\text{acMode/outputHomePower/ gridInputPower}$ abgeleitet), Seriennummer. Nicht erreichbare, nicht parsbare oder Geräte ohne Seriennummer werden übersprungen (Fehlerzähler $\text{connect/json/serial}$ pro Gerät), bleiben aber $\text{available} = \text{false}$ und fließen entsprechend nicht in die Verteilung ein.
5. $\text{calculate}()$ bildet die kombinierte Zielgröße:
 - sumZen = Summe der Ist-Leistung aller **verfügbaren** Geräte, dedupliziert nach ip .

- Sind gar keine Geräte verfügbar, wird der Zyklus ohne Schreibversuch beendet (“Kein Geraet erreichbar”).
 - $raw = round((gridPower - setpoint) + sumZen)$.
 - Dämpfung des **kombinierten** Werts auf `state.smoothedOutput` (siehe 8.2), daraus $target = round(state.smoothedOutput)$.
 - $target \geq 0$, und entweder bereits am Entladen ($sumZen > 0$) oder Bedarf $\geq dischargeStartupPower \rightarrow distributeDischarge(target)$ (Kapitel 6a). Sonst (Bedarf zu klein, noch nicht am Entladen) $\rightarrow$ alle Geräte auf 0 W (Anlauf-Totzone, systemweit, siehe Kapitel 8.9).
 - $target < 0$ und kein Gerät reverse-fähig $\rightarrow$ alle Geräte auf 0 W.
 - $target < 0$, mind. ein Gerät reverse-fähig, aber noch nicht am Laden ($sumZen \geq 0$) und Bedarf unterhalb `reverseStartupPower` $\rightarrow$ alle Geräte auf 0 W (Anlauf-Totzone, systemweit).
 - Sonst $\rightarrow distributeCharge(target)$ (Kapitel 6b).
6. `applyOutputs()` vergleicht für **jedes** Gerät den neuen Wert mit dem zuletzt geschriebenen `outputLimit`: Änderung $< hysteresis \rightarrow$ kein neuer Schreibvorgang für dieses Gerät. Zusätzlich wird geprüft, ob sich dabei die Richtung (`acMode`) ändert – falls ja und `directionChangeCooldown` seit dem letzten echten Richtungswechsel dieses Geräts noch nicht abgelaufen ist, wird die **alte** Richtung mit 0 W gehalten statt umzuschalten (siehe Kapitel 9.1). `dryRun`-Geräte werden nur geloggt (inkl. simulierter Cooldown-Zustand), nie tatsächlich geschrieben (siehe 4c).
 7. `writeAllDevices()` schreibt die verbleibenden Geräte **sequenziell** per HTTP POST `/properties/write(acMode, outputLimit/inputLimit, smartMode: 1)`; der Cooldown wird dabei mit einem frischen Zeitstempel unmittelbar vor dem eigentlichen Schreibvorgang nochmals geprüft. Erfolg/ Fehlschlag wird pro Gerät im Fehlerzähler `write` erfasst.
 8. Am Ende jedes Pfades wird `unlock()` aufgerufen: `state.busy = false`, Watchdog-Timer wird gelöscht.

8.2) Dämpfung des kombinierten Signals

```
state.smoothedOutput = state.smoothedOutput + dampingFactor * (raw - state.smoothedOutput)
```

- Die Dämpfung glättet das Reglerverhalten. Dies verhindert ein Über- oder Unterschwingen der Regelgröße. Die Dämpfung wirkt hier auf das **kombinierte** Roh-Signal (`raw`, aus Netzleistung + Summe aller Geräte), **bevor** dieses auf die einzelnen Geräte verteilt wird – nicht auf den Ausgabewert eines einzelnen Geräts. Ein Gerätewechsel im Konzentrationsmodus oder eine Kapazitätsgrenze eines einzelnen Geräts verändert also nicht die Glättung selbst, sondern nur, wie das bereits geglättete Ziel anschließend verteilt wird.
- `state.smoothedOutput` wird beim ersten Lauf nach Script-Start direkt auf den ersten berechneten `raw`-Wert gesetzt (kein Einschwingen von 0 aus nötig).
- `dampingFactor = 1.0` deaktiviert die Dämpfung faktisch (der Zielwert wird sofort 1:1 übernommen).

8.3) Watchdog / Timer

Zusammengefasst: Der Watchdog ist reine **Buchhaltung** auf Script-Ebene (“ich erlaube dem nächsten Zyklus zu starten”). Der Watchdog räumt ausschließlich den logischen Zustand des Scripts auf – nichts, was tatsächlich schon an die Firmware/das Netzwerk übergeben wurde. Empfehlung. Der Timer sollte mind. doppelt so hoch eingestellt sein, wie das Intervall

Bei `lock()` wird ein Timer über `watchdog ms` (Standard 10 s) gestartet. Seine Funktion: Er stellt

sicher, dass das Script **auch nach einem hängenden HTTP-Request wieder handlungsfähig ist**. Läuft der Timer ab, ohne dass `unlock()` zuvor aufgerufen wurde – z. B. weil ein Gerät nicht antwortet und der Callback nie feuert – wird `state.busy` zwangsweise auf `false` zurückgesetzt, ein Fehler vom Typ `watchdog` gemeldet (global, in `state.errors`), und der nächste reguläre Timer-Tick kann einen neuen Zyklus starten.

Da bei mehreren Geräten **sequenziell** gelesen und geschrieben wird (siehe 8.1 und Kapitel 10), muss `watchdog` den gesamten Zyklus abdecken – Netz lesen + alle Geräte lesen + Verteilung + alle Schreibvorgänge. Der separate `httpTimeout` (pro individuellem HTTP-Call) sollte deutlich kürzer als `watchdog` sein, damit jeder Einzel-Call sicher terminiert (erfolgreich oder mit Fehler), bevor der Gesamt-Watchdog überhaupt greifen könnte – so bleibt in der Praxis Zeit für einen sauberen, regulären Zyklusabbruch statt eines harten Watchdog-Resets.

Zusätzliche Absicherung im Multi-Script: Jeder asynchrone Callback (`Grid-Read`, `Geräte-Read`, `calculate()`, `applyOutputs()`, `writeDevice()`) prüft zu Beginn `myCycle !== state.cycleId` (`debugStale()`) und bricht ab, falls der Zyklus, zu dem der Callback gehört, inzwischen durch einen Watchdog-Reset oder einen neuen Tick überholt wurde. Das verhindert, dass eine verspätet eintreffende Antwort eines bereits abgebrochenen Zyklus noch Zustand eines **neueren** Zyklus überschreibt (siehe auch die „Bekannte Einschränkung“ in Kapitel 11).

Empfehlung für die Timer

Für intervall nicht unter 3 Sekunden gehen. Empfehlung für den Start

Anzahl Geräte	Bem.	intervall (s)	httptimeout (s)	watchdog (ms)
1		3000	5	6000
2		3500	5	7000
3	default	4000	5	10000
4		4500	5	10000

8.4) Fehlerzähler & Benachrichtigung

Anders als in der Einzelgerät-Version, wo alle sechs Fehlerarten in einem gemeinsamen Zähler-Objekt lagen, sind die Zähler hier in **global** und **pro Gerät** aufgeteilt:

Geltungsbereich	Typ	Ausgelöst durch
Global (<code>state.errors</code>)	<code>em</code>	Netzmesswert nicht verfügbar (<code>local: em:0</code> fehlt; <code>remote: Pro 3EM</code> nicht erreichbar/Antwort ungültig; <code>http_json: Meter</code> nicht erreichbar, Antwort nicht als JSON parsbar, oder konfiguriertes <code>gridSourceField</code> fehlt)
Global (<code>state.errors</code>)	<code>watchdog</code>	Zyklus durch Watchdog abgebrochen (siehe 8.3)
Pro Gerät (<code>ds.errors</code>)	<code>connect</code>	Gerät nicht erreichbar (HTTP-Fehler)
Pro Gerät (<code>ds.errors</code>)	<code>json</code>	Antwort des Geräts nicht als JSON parsbar
Pro Gerät (<code>ds.errors</code>)	<code>serial</code>	Keine Seriennummer in der Geräte-Antwort

Geltungsbereich	Typ	Ausgelöst durch
Pro Gerät (ds.errors)	write	Schreibvorgang an dieses Gerät fehlgeschlagen

Ablauf (identisch zur Einzelgerät-Version, jetzt pro Zähler-Objekt):

- Jeder Fehlschlag eines Typs erhöht dessen Zähler (reportError).
- Jeder Erfolg dieses Typs setzt den Zähler zurück (reportSuccess).
- Erreicht ein Zähler den konfigurierten errorThreshold (Standard: 5 aufeinanderfolgende Fehlschläge), wird **einmalig** eine Benachrichtigung gesendet – bei Geräte-Fehlern inkl. label des betroffenen Geräts. Erst wenn der Fehler behoben ist, wird bei erneutem Auftreten wieder benachrichtigt.
- Ist eine Benachrichtigung aktiv und der Fehler behebt sich, wird automatisch eine „behooben“-Meldung verschickt.

Dieses Schwellwert-Verfahren verhindert Alarm-Spam bei einzelnen, kurzzeitigen Netzwerk-Ausreißern und meldet nur anhaltende Probleme.

8.5) Signal-Benachrichtigung (CallMeBot) - inkl. WhatsApp

sendSignalMessage(text) sendet Nachrichten über die kostenlose CallMeBot-API per HTTP GET. Die Funktion ist inaktiv, solange CONFIG.signal.enabled = false gesetzt ist – die Kernregelung ist davon vollständig unabhängig. Sonderzeichen werden über simpleEncode() URL-kodiert (Leerzeichen, Umlaute, Satzzeichen etc.).

Neu in der Multi-Version: Über CONFIG.signal.typ lässt sich zwischen **SIGNAL** ODER **WHATSAPP** wählen.

```
signal: {
  enabled: true,
  typ: "SIGNAL",           // "SIGNAL" oder "WHATSAPP"
  phone: "PHONE-STRING",  // Telefonnummer bzw. Phone-ID von CallMeBot
  apiKey: "YOUR_API_KEY"
}
```

- typ: "SIGNAL" → Aufruf von <https://api.callmebot.com/signal/send.php?phone=...&apikey=...>
- typ: "WHATSAPP" → Aufruf von <https://api.callmebot.com/whatsapp.php?phone=...&text=...&> (Feldreihenfolge im Query-String unterscheidet sich leicht von Signal, ist aber im Script bereits korrekt so implementiert.)

Einrichtung (für beide Varianten identisch im Prinzip): Über [CallMeBot](#) wird ein kostenloser API-Key für den gewünschten Messenger angefordert:

1. Im Messenger (Signal **oder** WhatsApp) den jeweiligen CallMeBot-Kontakt anlegen bzw. die von CallMeBot vorgegebene Nummer speichern.
2. Die von CallMeBot vorgegebene Freigabenachricht an diesen Kontakt senden (für Signal z. B. *"I allow callmebot to send me messages"* – der exakte Text und die Kontaktnummer für WhatsApp können abweichen, siehe die aktuelle Anleitung auf callmebot.com, da sich Freigabetexte und IDs von Zeit zu Zeit ändern können).
3. Der Bot antwortet mit dem persönlichen apiKey.
4. phone (Telefonnummer bzw. die von CallMeBot zurückgegebene Phone-ID) und apiKey in CONFIG.signal eintragen, typ passend auf "SIGNAL" oder "WHATSAPP" setzen, enabled: true setzen.

Weiterführende, ausführliche Anleitung inkl. Screenshots zur CallMeBot- Einrichtung (am Beispiel eines anderen Scripts aus demselben Projekt, das Prinzip ist identisch): [Shelly-Script-Akkudaten-auslesen - Wiki](#)

Welche Nachrichten verschickt das Script:

- Einmalig beim Start (siehe 8.7): "Zendure Multi-Device-Controller gestartet (N Geräte)."
- Bei Erreichen von `errorThreshold` für einen Fehlertyp – global oder pro Gerät inkl. `label` (siehe 8.4).
- Als „behoben“-Meldung, sobald ein zuvor gemeldeter Fehler sich wieder erholt.

8.6) SoC-Grenzwert-Synchronisierung beim Start

`syncSocLimitsAll()` läuft **einmalig beim Script-Start**, bevor der reguläre Regelbetrieb (der `interval-Timer`) startet, und schreibt für **jedes** konfigurierte Gerät (außer `dryRun`-Einträge, die dabei nur geloggt werden) die konfigurierten `minSoc`/`maxSoc`-Werte direkt in die Firmware-eigenen SoC-Grenzwert-Properties des Zendure-Geräts:

- `minSoc` (Script) → `minSoc-Property` (Zendure, ×10, z. B. 15 % → 150)
- `maxSoc` (Script) → `socSet-Property` (Zendure, ×10, z. B. 100 % → 1000)

8.7) Startprotokoll (Banner)

Beim Scriptstart werden alle aktiven Konfigurationswerte einmalig ins Log geschrieben – u. a. Geräteliste (Array-Index als `[devN]`, Label, IP, Entladen-Status (`dischargeAllowed`), `minSoc`, `maxOutput`, Lade-vom-Netz-Status inkl. `maxInputPower`/`maxSoc`, `DRYRUN`-Kennzeichnung), Grid-Quelle (bei `remote` inkl. IP, bei `http_json` inkl. URL/Feldname/`Invert-Status`), Intervall, Watchdog, HTTP-Timeout, Sollwert, Hysterese, Dämpfungsfaktor, Verteil-Schwellen für Entladen und Laden, Ausgleichs-Schwelle, Reverse-Start/Stop-Werte, Discharge-Start/Stop-Werte (Kapitel 8.9), Richtungswechsel-Cooldown (Kapitel 9.1), Fehlerschwelle, Debug- und Signal-Status. Der `[devN]`-Index im Banner entspricht dem Array-Index in `CONFIG.devices` und damit dem N in den per-Geräte-KVS-Keys (`zdmc_devN_...`, siehe Kapitel 13). Die Ausgabe erfolgt zeilenweise über eine leicht zeitversetzte Timer-Kette (`printBannerLine`, 150 ms Abstand), um Truncation/Verlust einzelner Zeilen in der Shelly-Konsole zu vermeiden.

Reihenfolge beim Start:

1. Banner wird zeilenweise ausgegeben.
2. Ist `signal.enabled` aktiv, wird eine Start-Nachricht verschickt ("... gestartet (N Geräte).").
3. `syncSocLimitsAll()` läuft einmalig über alle Geräte (siehe 8.6).
4. Erst danach startet der reguläre `interval-Timer` und damit der eigentliche Regelbetrieb.

8.8) Zeitlich gekoppelte Hysterese (`spread` → `single`)

Die reine Leistungs-Hysterese aus Kapitel 7 (`concentrateBelow/spreadAbove`) prüft nur den *Momentanwert* des kombinierten Ziels. Bei periodisch schwankenden Lasten knapp um diese Schwellen (Wolken/Sonne-Wechsel, taktende Großverbraucher wie eine Heißluftfritteuse) reicht das allein nicht aus – das Ziel pendelt dann ständig über und unter `concentrateBelow`, und das Script würde ohne weitere Absicherung ebenso ständig zwischen Konzentrations- und Verteilmodus hin- und herschalten (häufigere Schreibvorgänge, unruhigeres Verhalten der Geräte).

Deshalb ist der Wechsel **`spread` → `single`** zusätzlich an eine Haltezeit gekoppelt; der Wechsel **`single` → `spread`** bleibt bewusst sofort, da ein kurzfristig steigender Bedarf möglichst schnell auf alle Geräte verteilt werden soll.

Zustand: Jede Richtung führt einen eigenen Zähler: `state.discharge.holdCycles` bzw. `state.charge.holdCycles`.

Umrechnung Minuten → Zyklen: Einmalig beim Start berechnet:

`CONCENTRATE_HOLD_CYCLES = max(1, round(concentrateHoldMinutes * 60000 / interval))`

Bei den Standardwerten (`concentrateHoldMinutes: 3`, `interval: 4000`) ergibt das `CONCENTRATE_HOLD_CYCLES = 45` Zyklen.

Zusammenspiel mit dem Kapazitäts-Override: Reicht das aktuell aktive Einzelgerät (`maxOutput/maxInputPower`) für das Ziel nicht mehr aus, wird weiterhin **sofort** in den Verteilmodus gewechselt (siehe Kapitel 6a/6b) – unabhängig von der Haltezeit-Logik. Auch dabei wird `holdCycles` auf 0 zurückgesetzt.

Beispiel (`concentrateBelow: 2000`, `spreadAbove: 2400`, `concentrateHoldMinutes: 3`, `interval: 4000` → 45 Zyklen Haltezeit):

Ziel fällt von 2600 W auf 1800 W und bleibt dort für 2 Minuten (30 Zyklen), steigt kurz auf 2100 W (Totzone – Zähler friert bei 30 ein, kein Reset), fällt danach wieder auf 1800 W zurück: Der Zähler zählt ab hier weiter bei 30 – nach weiteren 15 Zyklen (45 insgesamt) erfolgt der Wechsel auf "single". Wäre das Ziel stattdessen kurz auf 2500 W gestiegen (über `spreadAbove`), hätte der Zähler auf 0 zurückgesetzt und wäre komplett neu gestartet.

8.9) Start/Stop-Hysterese (Laden & Entladen)

Sowohl Laden als auch Entladen kennen je **zwei globale Schwellen** – eine Start- und eine (niedrigere) Stop-Schwelle. Beide Schwellenpaare sind nach demselben Prinzip aufgebaut, wirken aber auf zwei unterschiedlichen Ebenen:

Schwelle	Wert (Standard)	Ebene
<code>dischargeStartupPower</code>	35 W	Aggregat – einmal pro Zyklus, auf die Summe aller Geräte
<code>dischargeStopPower</code>	15 W	Pro Gerät/Geräteanteil – jedes Mal, für jedes Gerät einzeln
<code>reverseStartupPower</code>	30 W	Aggregat – einmal pro Zyklus, auf die Summe aller Geräte
<code>reverseStopPower</code>	10 W	Pro Gerät/Geräteanteil – jedes Mal, für jedes Gerät einzeln

Die Start-Schwelle wirkt nur einmal, ganz oben in `calculate()`:

```
let alreadyDischarging = sumZen > 0;
if (!alreadyDischarging && target < CONFIG.dischargeStartupPower) {
  output = zeroOutputs(); // Anlauf-Totzone: gar nicht erst starten
} else {
  output = distributeDischarge(target);
}
```

Sie prüft die **Gesamtsumme** (target), nicht den Anteil eines einzelnen Geräts. Läuft die Entladung bereits (sumZen > 0, aus der zuletzt gemessenen Ist-Leistung), greift die Start-Schwelle gar nicht mehr – dann zählt nur noch die (niedrigere) Stop-Schwelle.

Die Stop-Schwelle wirkt dagegen wiederholt, für jedes Gerät einzeln – im Konzentrationsmodus direkt in distributeDischarge(), im Verteilmodus am Ende von waterFillDischarge() (analog waterFillCharge() auf der Ladeseite):

```
if (o < CONFIG.dischargeStopPower) {
  o = 0; // Anteil zu klein -> AUS, nicht auf einen Mindestwert angehoben
}
```

Das ersetzt das frühere pro-Gerät minOutput: Ein Gerät läuft entweder mit sinnvoller Leistung oder gar nicht, nie dauerhaft mit winzigen, ineffizienten Werten.

Beispiel - ein Gerät, Entladen (dischargeStartupPower: 35, dischargeStopPower: 15):

Zyklus	Ziel	Zustand vorher	Ergebnis	Warum
1	30 W	aus	0 W	unter Start-Schwelle (35 W), noch nicht aktiv → bleibt aus
2	40 W	aus	40 W	≥ Start-Schwelle (35 W) → läuft sofort mit vollem Zielwert an
3	20 W	aktiv (40 W)	20 W	bereits aktiv, 20 W ≥ Stop-Schwelle (15 W) → bleibt an
4	10 W	aktiv (20 W)	0 W	bereits aktiv, aber 10 W < Stop-Schwelle (15 W) → schaltet ab
5	30 W	aus	0 W	wieder aus dem Stand: 30 W < Start-Schwelle (35 W) → bleibt aus

Auffällig: Zyklus 3 (20 W) und Zyklus 5 (30 W) erhalten trotz höheren Zielwerts unterschiedliche Ergebnisse – reiner Beleg der Start≠Stop-Hysterese (Zyklus 3 profitiert davon, dass das Gerät bereits lief).

Dieselbe Logik gespiegelt für Laden (reverseStartupPower: 30, reverseStopPower: 10) – nur mit umgekehrtem Vorzeichen und maxInputPower statt maxOutput als Kappungsgrenze; siehe Kapitel 6b.

Verteilmodus - ein wichtiger Randfall: Die Stop-Schwelle wird **pro Geräteanteil** geprüft, nicht auf die Summe. Sind mehrere Geräte gleichzeitig aktiv (Verteilmodus) und der jeweilige Anteil jedes einzelnen Geräts liegt unter der Stop-Schwelle, werden **alle** betroffenen Geräte auf 0 gesetzt – auch wenn die Summe aller Anteile noch über der Start-Schwelle läge. Das ist bewusst so (identisch zum bestehenden Verhalten von `waterFillCharge()`): kein Sonderfall zwingt ein einzelnes Gerät zu einer Mindestleistung, wenn sein fairer Anteil eigentlich zu klein wäre.

8.10) Logausgaben

Hier ist die Übersicht aller im Skript definierten Log-Ausgaben, unterteilt nach ihrem Verhalten im Normalbetrieb, bei Statusänderungen, im Simulationsmodus (`dryRun`), im Debug-Modus und im Fehlerfall:

Kategorisierung	Log-Ausgabe in der Konsole	Bedingung / Auslöser
Standard (Jeder Zyklus)	Grid: [Power] W Summe Geraete: [Power] W Kombiniertes Ziel (gedaempft): [Target] W [Label]: Leistung gesetzt: [Power] W (Export/Laden vom Netz)	Standard in jedem erfolgreichen Zyklus. Wird am Ende der Berechnung in <code>calculate()</code> ausgegeben. Ein Sollwert wurde erfolgreich via HTTP-POST an das Gerät übermittelt (Abweichung > Hysterese). [Power] ist der effektiv geschriebene Wert – bei aktivem <code>directionChangeCooldown</code> kann das 0 W sein, auch wenn der berechnete Zielwert ungleich 0 war (siehe Kapitel 9.1).
Schreibvorgang		
Vorschau / Simulation	[Label]: [DRYRUN] wurde schreiben: [Power] W (Export/Laden vom Netz)	Gerät ist auf <code>dryRun: true</code> gesetzt. Der Schreibvorgang wird nur simuliert – inkl. simuliertem <code>Cooldown</code> -Zustand ([Power] berücksichtigt bereits einen ggf. aktiven <code>Cooldown</code>).
Vorschau (Zeile 2)	[Label]: SOC [SoC]% Ist [Power] W Soll [Power] W <code>acMode</code> [Mode] ([ModeText]) [Cooldown haelt Richtung] [DRYRUN...]	Wird in <code>applyOutputs()</code> vor jedem möglichen Schreibvorgang für jedes Gerät in der Konsole ausgegeben. Das optionale [Cooldown haelt Richtung] erscheint nur, wenn <code>directionChangeCooldown</code> den ursprünglich berechneten Richtungswechsel für dieses Gerät in diesem Zyklus zurückhält.

Kategorisierung	Log-Ausgabe in der Konsole	Bedingung / Auslöser
Logik: Cooldown	Cooldown aktiv ([N]s verbleibend) - Richtungswechsel blockiert, halte [ModeText]	CONFIG.debug: true ist aktiv und directionChangeCooldown blockiert für dieses Gerät in diesem Zyklus einen Richtungswechsel (Laden↔Entladen). Siehe Kapitel 9.1.
Logik: Moduswechsel	Ziel uebersteigt maxOutput von [Label] - wechsele sofort in den Mehrere-Geraete-Modus	Das Leistungsziel ist höher als die maximale Entladeleistung des aktiven Einzelgeräts.
Logik: Moduswechsel	Ladebedarf uebersteigt maxInputPower von [Label] - wechsele sofort in den Mehrere-Geraete-Modus	Der Ladebedarf ist höher als die maximale Ladeleistung des aktiven Einzelgeräts.
Logik: Rebalancing	Ausgleich: bevorzugtes Geraet wechselt zu [Label] (Vorsprung [N] Prozentpunkte)	Ein anderes Gerät hat beim SoC einen Vorsprung, der die Schwelle rebalance.socMargin erreicht/übersteigt.
Status: SoC-Limit	[Label]: SOC-Obergrenze erreicht ([SoC]% >= [MaxSoc]%) - Laden vom Netz gesperrt	SoC des Geräts hat maxSoc erreicht. Laden über das Netz wird gesperrt (wird nur einmalig bei Erreichen geloggt).
Status: SoC-Limit	[Label]: SOC wieder unter Obergrenze ([SoC]% < [MaxSoc]%) - Laden bei Bedarf wieder moeglich	SoC ist wieder unter maxSoc gefallen (wird nur einmalig bei Unterschreitung geloggt).
Fehler: Abbruch	Kein Geraet erreichbar - Zyklus uebersprungen	Im aktuellen Zyklus konnte keines der konfigurierten Geräte erfolgreich abgefragt werden.
Fehler: Überwachung	FEHLER ([Label]/[Type]): [Message] - aufeinanderfolgende Fehler: [Count]	Ein HTTP-, JSON-, Seriennummern- oder Watchdog-Fehler ist aufgetreten.
KVS (Live-Overrides)	KVS [Key]: Wert uebernommen ([Old] -> [New])	kvsEnabled: true, debug: true und ein KVS-Key wurde erfolgreich geändert und geladen.
Debug: Zyklus-Start	DEBUG Zyklus [ID] gestartet	CONFIG.debug: true ist aktiv beim Starten eines neuen Zyklus.
Debug: Zyklus-Ende	DEBUG Zyklus [ID] abgeschlossen nach [MS] ms	CONFIG.debug: true ist aktiv beim erfolgreichen Abschluss eines Zyklus.

Kategorisierung	Log-Ausgabe in der Konsole	Bedingung / Auslöser
Debug: HTTP POST	DEBUG httpPost -> url: [URL] body: [Body]	CONFIG.debug: true ist aktiv vor jedem HTTP-POST-Request.

9) Ablauf der Regelung (Diagramm)

(Das Diagramm ist aus Platzgründen auf den folgenden drei Seiten abgebildet.)

9.1) Cooldown-Timer für Richtungswechsel (Beispiel)

Das Diagramm oben endet mit dem eigentlichen Schreibvorgang – **ein** Zwischenschritt ist darin bewusst nicht im Detail dargestellt, um es nicht zu überladen: die Cooldown-Prüfung auf ac-Mode-Ebene direkt vor Y bzw. AB („Schreiben: POST an Gerät“). Dieses Unterkapitel erklärt sie im Detail.

Warum überhaupt nötig: Reine AC-ladefähige Solarflows (ohne eigene smarte Umschaltrampe) können bei sehr schnellen Vorzeichenwechseln des Smartmeters (z. B. Sekundenbruchteil-Sprung von deutlichem Import auf deutlichen Export) unnötig oft zwischen Laden und Entladen hin- und herschalten. `directionChangeCooldown` bremst **nur** diesen Richtungswechsel (`acMode 1↔2`) – Leistungsänderungen **innerhalb** derselben Richtung sind davon nicht betroffen und laufen weiterhin sofort (nur über die normale hysteresis gefiltert).

Zwei Werte pro Gerät, intern gepflegt:

- `ds.acMode` – die zuletzt **tatsächlich geschriebene** (oder bei `dryRun` simulierte) Richtung.
- `ds.acModeChangedAt` – Zeitpunkt des letzten **echten** Wechsels dieser Richtung.

Ablauf, wenn ein Richtungswechsel ansteht:

1. Die Verteilung (Kapitel 6/8.1) berechnet wie gewohnt den rohen Zielwert für ein Gerät (`rawPlan`), unabhängig vom Cooldown.
2. `enforceDirectionCooldown()` vergleicht `rawPlan.acMode` mit `ds.acMode`:
 - **Gleich** → nichts passiert, der Zielwert geht unverändert durch.
 - **Unterschiedlich** und seit `ds.acModeChangedAt` sind weniger als `directionChangeCooldown` ms vergangen → der Plan wird überschrieben: **alte** Richtung (`ds.acMode`) wird beibehalten, aber mit **0 W**. Es erfolgt **kein** `acMode`-Wechsel am Gerät.
 - **Unterschiedlich** und die Sperrzeit ist abgelaufen → der rohe Plan geht unverändert durch, **direkt mit dem vollen aktuellen Zielwert** der neuen Richtung (kein schrittweises Hochfahren).

Beispiel (`interval`: 4000 ms, `directionChangeCooldown`: 8000 ms – Faktor 2, siehe Faustregel unten):

Zeit	Ziel (roh)	tatsächlich geschrieben	<code>acModeChangedAt</code>
<code>t = 0 s</code>	+500 W (Entladen)	<code>acMode 2, 500 W</code>	wird auf 0 s gesetzt
<code>t = 4 s</code>	–500 W (Laden)	<code>acMode 2 gehalten, 0 W</code> ← Cooldown blockiert (<code>4 s < 8 s</code>)	bleibt bei 0 s
<code>t = 8 s</code>	–500 W (Laden)	<code>acMode 1, –500 W</code> ← Sperrzeit abgelaufen (<code>8 s ≥ 8 s</code>), sofort voller Wert	wird auf 8 s gesetzt

Zwischen `t=4s` und `t=8s` wird effektiv nur **einmal** geschrieben (der Übergang auf 0 W bei `t=4s`) – der eigentliche Richtungswechsel bei `t=8s` wird trotz unverändertem Zielwert **nicht** durch die normale hysteresis gefiltert, weil sich der effektiv geschriebene Wert dabei wirklich ändert (0 W → –500 W).

Dimensionierung – Verhältnis zu `interval`: Die Sperrzeit wird nur bei jedem Poll geprüft (Vielfache von `interval`). Ist `directionChangeCooldown` zu klein gewählt, hat sie **keine** Wirkung, da sie beim nächstfolgenden Poll bereits abgelaufen wäre. Faustregel:

Anzahl geblockter Zyklen $\approx \text{floor}(\text{directionChangeCooldown} / \text{interval})$

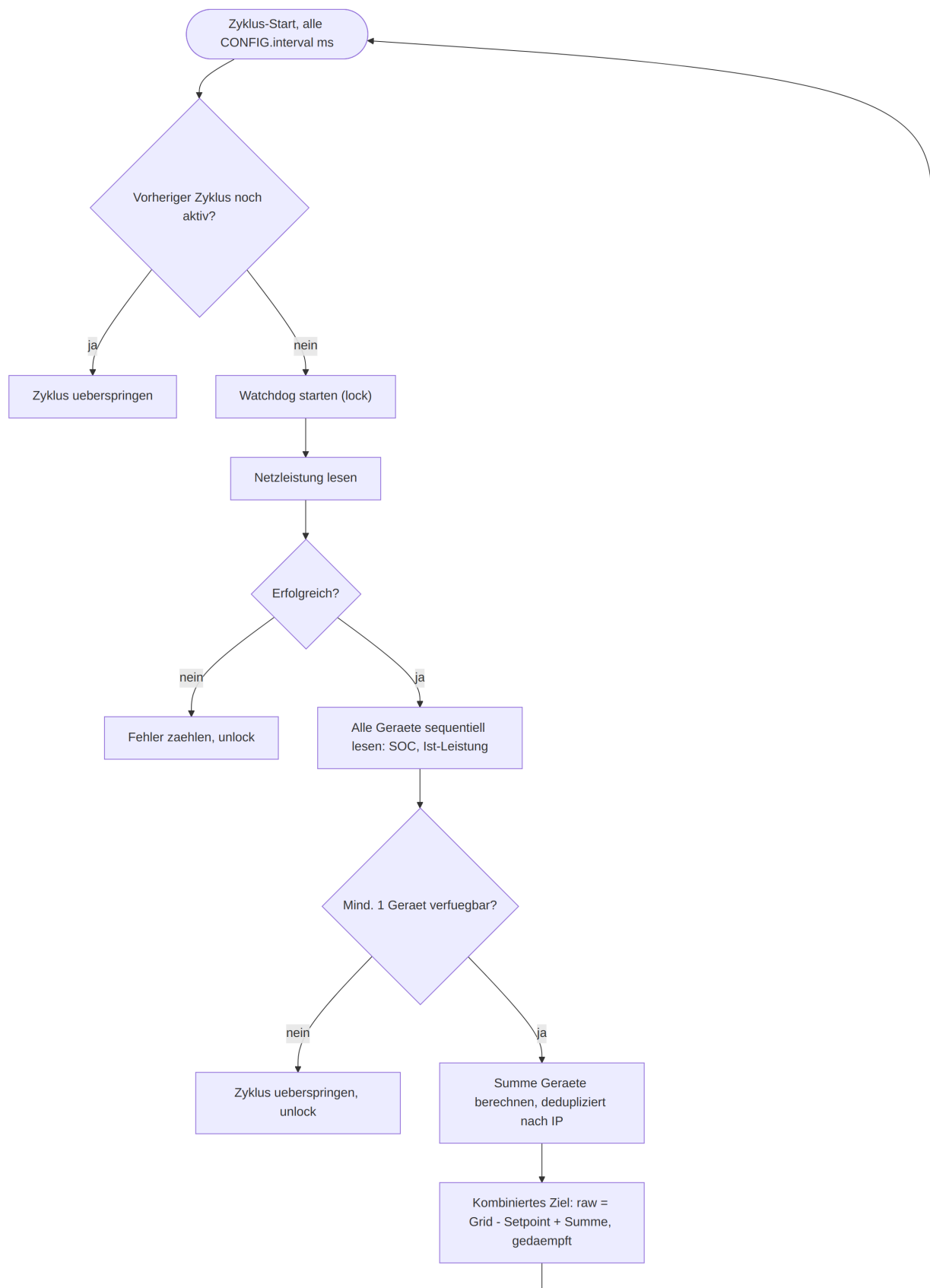


Figure 1: Ablaufdiagramm Teil 1/3

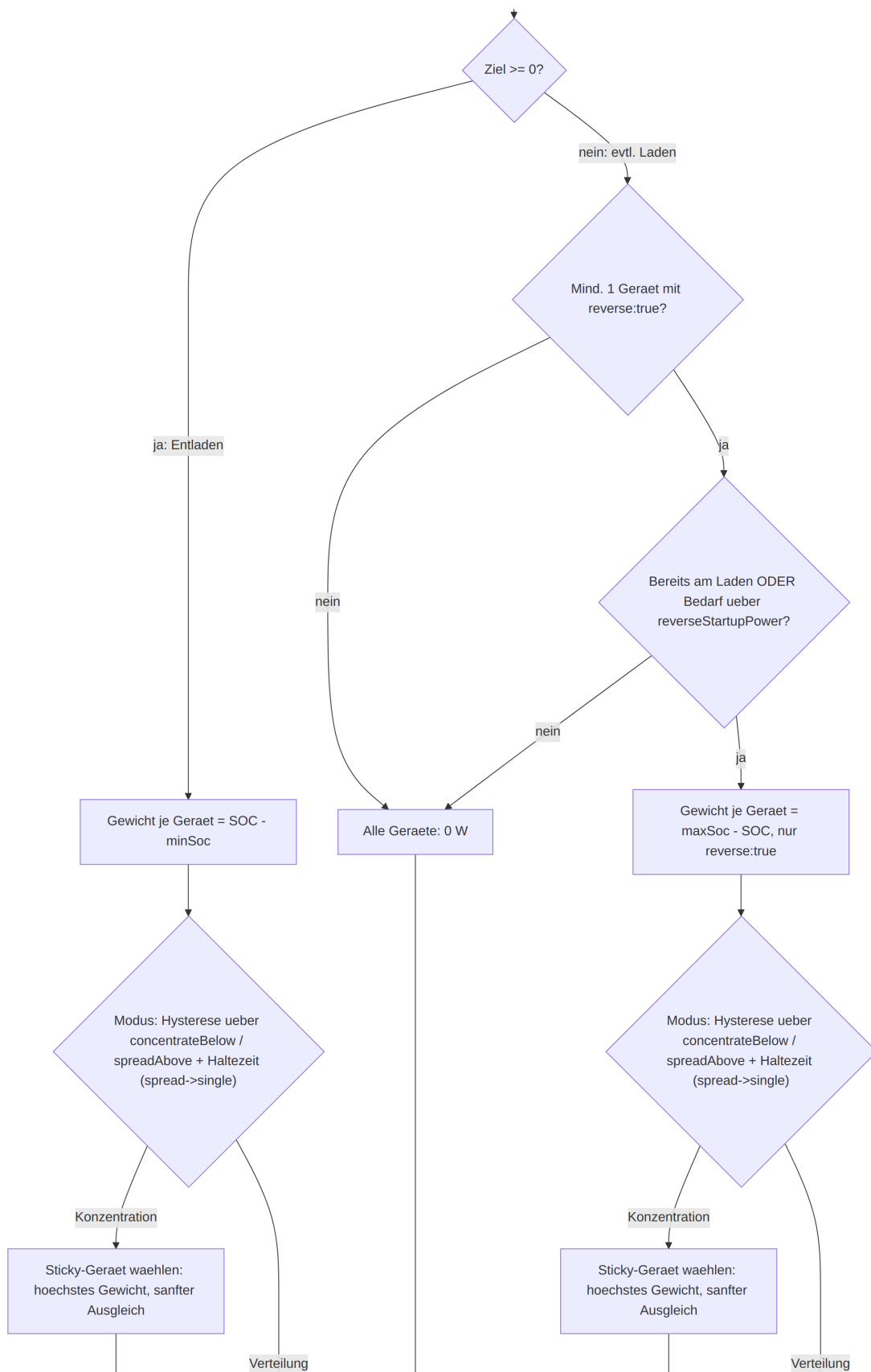


Figure 2: Ablaufdiagramm Teil 2/3

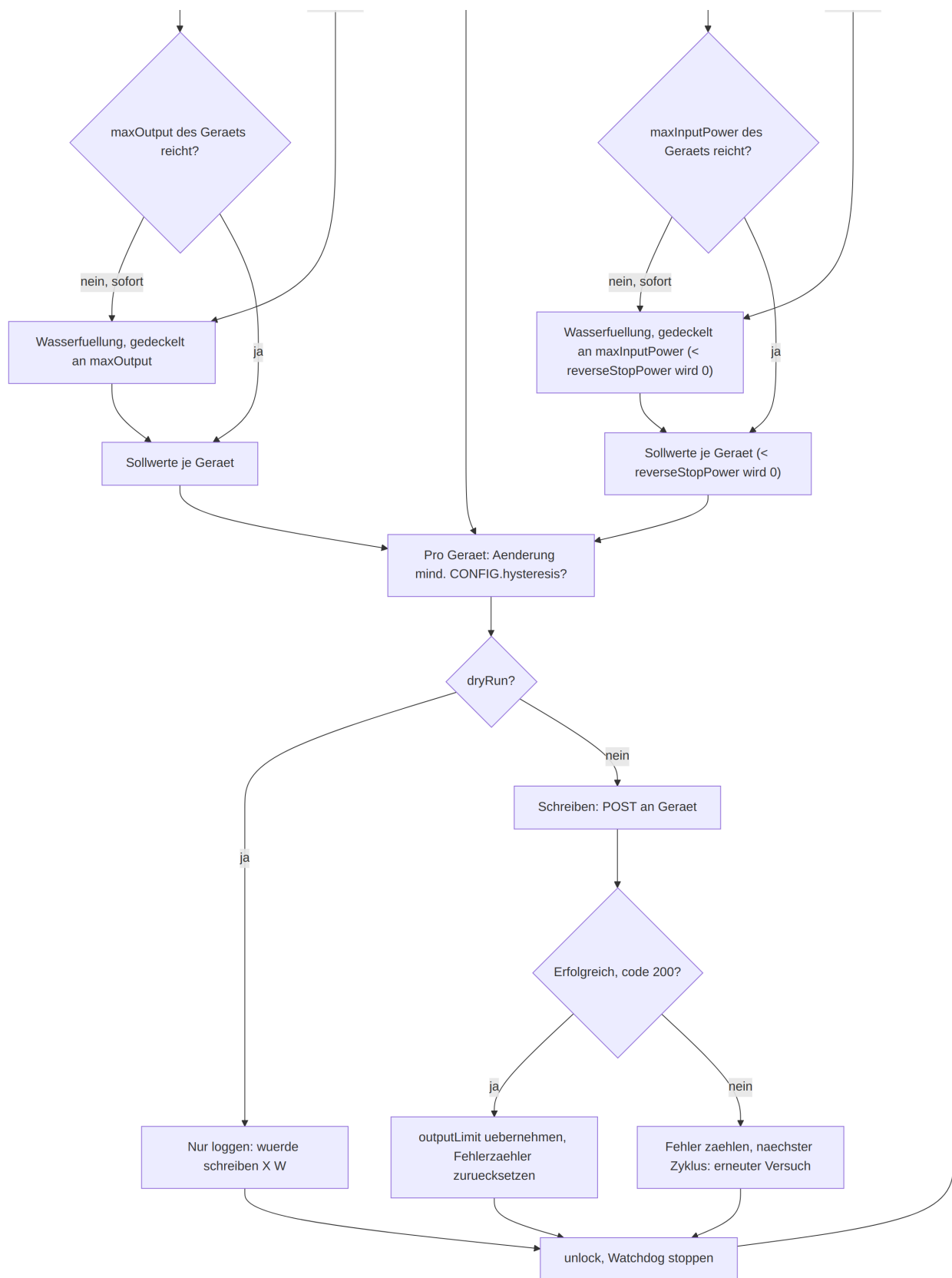


Figure 3: Ablaufdiagramm Teil 3/3

Bei `interval: 4000` und `directionChangeCooldown: 8000` (Faktor 2) wird also **genau 1 voller Zyklus** geblockt, der Wechsel selbst erfolgt beim zweiten Poll nach dem erkannten Flip. Die Validierung in Kapitel 4a stellt lediglich sicher, dass `directionChangeCooldown` **mindestens** $1.5 \times \text{interval}$ beträgt (≥ 1 blockierter Zyklus); für mehr Sicherheitsabstand (z. B. 2 volle Zyklen) `directionChangeCooldown` entsprechend höher wählen, z. B. $\text{interval} \times 3$ plus etwas Marge, um die Gleichheits-Grenze sauber zu vermeiden.

Wichtig, was der Cooldown NICHT tut: Er sieht den eigentlichen Sekundenbruchteil-Flip beim Smartmeter selbst nie – der liegt schneller als die Abtastrate des Scripts. Er verhindert stattdessen, dass das Script bei mehreren aufeinanderfolgenden, aber echten Polls mit wechselndem Vorzeichen jedes Mal sofort real umschaltet.

10) Hinweis: Performance, Grenzen und Status-Prüfung des Shelly

Performance & Grenzen

- Community-Script, kein offizielles Zendure- oder Shelly-Produkt. Keine Garantie für Funktion oder Genauigkeit – Einsatz auf eigenes Risiko, insbesondere bei automatisierter Steuerung realer Ladeleistungen.
- Shelly-Scripts laufen im selben Task wie die restliche Geräte-Firmware und teilen sich dessen Rechenzeit – sie dürfen nicht lange blockieren. Andere parallel laufende Funktionen auf demselben Gerät (Energiesmessung, Cloud-Verbindung, weitere Scripts) konkurrieren um dieselben Ressourcen.
- Begrenzter Script-Speicher (in unseren Tests ca. 25 KB gesamt pro Script, geräte-/firmwareabhängig) und begrenzte Aufruf-Tiefe – bei vielen Geräten oder eigenen Erweiterungen der Logik kann das an Grenzen stoßen (in diesem Script bereits durch gezielte `Timer.set(0, ...)`-Sprünge abgefangen, siehe Testverlauf).
- Maximal 3 Scripts gleichzeitig laufend pro Shelly-Gerät.
- Alle Geräte-Anfragen laufen bewusst sequentiell, nicht parallel – die Zykluszeit steigt linear mit der Anzahl der Geräte. Bei vielen Geräten `CONFIG.interval` und `CONFIG.watchdog` ggf. großzügiger wählen.
- Kein Echtzeitverhalten garantiert – WLAN-Aussetzer, Netzwerklatenz und reale Rampezeiten der Wechselrichter können zu spürbaren Verzögerungen führen.
- Interner Zustand (aktives Gerät, Fehlerzähler, Glättung) geht bei einem Geräte-Neustart verloren und beginnt bei null – keine Persistenz über einen Neustart hinweg.

10.1) Debug

Befehle zur Statusprüfung (im Browser aufrufen, <IP> durch die Shelly-Adresse ersetzen):

Zweck	Befehl
Laufende Scripts auflisten (IDs, Name, Status)	<code>http://<IP>/rpc/Script.List</code>
Speicherverbrauch eines bestimmten Scripts	<code>http://<IP>/rpc/Script.GetStatus?id=<ID></code>
Geräte-Betriebszeit, RAM des Gesamtsystems	<code>http://<IP>/rpc/Sys.GetStatus</code>

Debugmodus aktivieren

- 1) Parameter `debug: true` setzen

Script starten

- 2) Auswerten
 - zeigt alle Initialisierungsparameter

- zeigt Verteilungslogik
- zeigt Zykluszeiten
- zeigt REST-Call Aufrufe
- alle Zeilen mit "DEBUG" am Anfang werden nur mit debug: true ausgegeben
- Fehlersituationen und Verworfenen Zyklen werden sichtbar(er)

anbei ein Auszug

```
-----
Verion 2.0.0 KVS (direction change cooldown)
Zendure Multi-Device Controller gestartet
Geraete      : 2
  - [dev0] SF2400 (192.168.178.143): Entladen ja, minSoc 18%, maxOutput 800 W, Laden vom Netz
  - [dev1] Fatamorgana (192.168.178.143): Entladen nein, minSoc 15%, maxOutput 800 W, Laden vom Netz
Grid source: local
Interval     : 4000 ms
Watchdog     : 10000 ms
HTTP-Timeout: 5 s (pro Anfrage)
Setpoint     : 0 W
Hysteresis   : 10 W (pro Geraet)
Damping      : 0.6
Entladen     : ein Geraet unter 600 W, verteilen ueber 800 W
Laden        : ein Geraet unter 600 W, verteilen ueber 800 W
Konzentrieren-Haltezeit: 3 min (45 Zyklen) - gilt fuer spread->single, discharge+charge
Ausgleich    : ab 10 Prozentpunkten Vorsprung, sofort
Reverse Start/Stop: 30 W / 10 W
Discharge Start/Stop: 35 W / 15 W
Richtungswechsel-Cooldown: 8 s (pro Geraet)
Err.Thresh   : 5
Debug        : aktiviert
Signal       : deaktiviert
KVS-Feature: DEAKTIVIERT (kein Live-Override, kein Seeding)
-----
Synchronisiere SoC-Grenzwerte (minSoc/maxSoc) einmalig mit allen Geraeten...
DEBUG httpPost -> url: http://192.168.178.143/properties/write | body: {"sn":"XXXXXXX","prop":{
  SF2400: SoC-Grenzwerte synchronisiert (minSoc 18%, maxSoc 100%)
  Fatamorgana: [DRYRUN] SoC-Grenzwerte werden nicht geschrieben
SoC-Sync abgeschlossen.
Pruefe KVS auf fehlende Default-Werte (einmalig)...
KVS-Seed uebersprungen - kvsEnabled: false
KVS-Seed abgeschlossen - starte Regelbetrieb.
-----
DEBUG Zyklus 1 gestartet
Grid: 520 W | Summe Geraete: 0 W | Kombiniertes Ziel (gedaempft): 520 W
  SF2400: SOC 62% | Ist 0 W | Soll 520 W | acMode 2 (Export)
  Fatamorgana: SOC 34% | Ist 0 W | Soll 0 W | acMode 1 (Import/Idle) [DRYRUN -
wird nicht geschrieben]
  Fatamorgana: [DRYRUN] wuerde schreiben: 0 W (Laden vom Netz)
DEBUG httpPost -> url: http://192.168.178.143/properties/write | body: {"sn":"XXXXXXX","prop":{
SF2400: Leistung gesetzt: 520 W (Export)
DEBUG Zyklus 1 abgeschlossen nach 1180.4 ms
DEBUG Zyklus 2 gestartet
Grid: -480 W | Summe Geraete: 520 W | Kombiniertes Ziel (gedaempft): -68 W
```

Cooldown aktiv (4s verbleibend) - Richtungswechsel blockiert, halte Export

SF2400: SOC 62% | Ist 520 W | Soll -68 W | acMode 2 (Export) [Cooldown haelt Richtung]

DEBUG httpPost -> url: http://192.168.178.143/properties/write | body: {"sn":"XXXXXXX","pro

SF2400: Leistung gesetzt: 0 W (Export)

DEBUG Zyklus 2 abgeschlossen nach 980.2 ms

Im Beispiel wechselt das Grid-Signal von starkem Export- auf Ladebedarf (Zyklus 1 → 2). SF2400 hatte gerade erst umgeschaltet (acModeChangedAt bei $t \approx 0$), daher blockiert directionChangeCooldown den sofortigen Rückwechsel auf Laden in Zyklus 2 – das Gerät hält Export bei 0 W statt direkt auf Laden vom Netz zu springen (siehe Kapitel 9.1 für den vollen zeitlichen Ablauf).

11) Bekannte Einschränkung

Da HTTP-Aufrufe asynchron laufen, kann es grundsätzlich zu einer Race Condition zwischen einem Watchdog-Reset und einem noch laufenden HTTP-Callback kommen (der Callback wirkt sich ggf. auf einen bereits neu gestarteten Zyklus aus). Im Multi-Device-Script wird dem bereits aktiv entgegengewirkt: Jeder relevante Callback prüft `myCycle !== state.cycleId` (`debugStale()`) und verwirft veraltete Antworten, statt sie auf einen inzwischen neueren Zyklus anzuwenden (siehe Kapitel 8.3/8.1). Damit ist die in der Einzelgerät-Version noch offene Einschränkung hier größtenteils adressiert – ausgeschlossen ist eine Restunschärfe bei sehr knapp getakteten `httpTimeout/watchdog`-Kombinationen dennoch nicht vollständig, weshalb `httpTimeout` deutlich kürzer als `watchdog` gewählt werden sollte (siehe 8.3).

12) getestete Beispielkonfiguration

```
devices: [
{
  ip: "192.168.68.XXX", // Zendure IP address
  label: "SF2400AC1", // short name, used in logs/messages

  minSoc: 15,           // no discharge below this SOC (%)
  maxOutput: 2400,      // max discharge/export power (W)
  dischargeAllowed: true, // may this device discharge/export at all?

  reverse: true,        // may this device charge from the grid?
  maxSoc: 100,          // no charging from grid at/above this SOC (%)
  maxInputPower: 2400,  // max charge power from grid (W)

  dryRun: false        // true = read + calculate only, never write
},
{
  ip: "192.168.68.XXX",
  label: "SF2400AC2",

  minSoc: 15,
  maxOutput: 2400,
  dischargeAllowed: true,

  reverse: true,        // this device is never charged from the grid
  maxSoc: 100,
```

```

    maxInputPower: 2400,

    dryRun: false
},
{
    ip: "192.168.68.XXX",
    label: "SF2400Pro",

    minSoc: 15,
    maxOutput: 2400,
    dischargeAllowed: true,

    reverse: true,           // this device is never charged from the grid
    maxSoc: 100,
    maxInputPower: 2400,

    dryRun: false
},

interval: 3000,
watchdog: 8000,
httpTimeout: 5,
errorThreshold: 5,

setpoint: -40,
hysteresis: 40,
dampingFactor: 0.6,
discharge: {
    concentrateBelow: 2000, // W - below this combined target, use ONE device
    spreadAbove: 2400 // W - above this, split across all devices
},

charge: {
    concentrateBelow: 2000,
    spreadAbove: 2400
},

rebalance: {
    socMargin: 10, // percentage points of advantage required
}

concentrateHoldMinutes: 3,

reverseStartupPower: 30,
reverseStopPower: 10,

dischargeStartupPower: 35,
dischargeStopPower: 15,

directionChangeCooldown: 8000,

```

13) Live-Override der Regelparameter per KVS (z. B. Home Assistant)

KVS = Key-Value Store

...ist gedacht für User, die Einfluss auf die Regelparameter nehmen wollen, ohne die Shelly Konfiguration im Script erneuern zu müssen, sozusagen "online" "on the fly"

[Script herunterladen](#)

Drei globale Werte plus zwei Werte **pro Gerät** lassen sich **ohne Neustart und ohne Code-Änderung** von außen überschreiben – über den eingebauten Key-Value-Store (KVS) des Shelly. Das Script liest die passenden Keys zu Beginn jedes Zyklus und übernimmt gültige Werte sofort in CONFIG; ist KVS nicht erreichbar oder ein Key ungültig (außerhalb des Gültigkeitsbereichs), bleibt der zuletzt gültige Wert unverändert – die Regelung läuft immer unbeeinflusst weiter.

Zwei zusätzliche Steuerparameter im Script (CONFIG, nicht per KVS überschreibbar)

Neben den fünf live-überschreibbaren Werten unten gibt es zwei Schalter, die das KVS-Feature selbst steuern. Diese werden **nur im Script** (CONFIG-Block, siehe auch Kapitel 4) gesetzt, nicht per KVS.Set von außen:

CONFIG-Parameter	Standard	Wirkung
kvsEnabled	true	Hauptschalter für das gesamte Feature. false deaktiviert KVS.GetMany pro Zyklus und das Seeding beim Start komplett – alle fünf Werte unten sind dann fest auf die CONFIG-Werte eingefroren, ein KVS.Set von außen bleibt wirkungslos. Sinnvoll, wenn niemand (Home Assistant, curl, ...) diese Keys tatsächlich nutzt, um den zusätzlichen lokalen RPC-Call pro Zyklus zu sparen.

kvsForceReseed	false	Nur relevant bei kvsEnabled: true. Steuert das Verhalten beim Start : false (Standard) befüllt nur fehlende Keys mit den CONFIG-Werten, vorhandene (auch live gesetzte) Werte bleiben unangetastet. true überschreibt bei jedem Start alle Keys mit den aktuellen CONFIG-Werten – z. B. nützlich, um nach einer CONFIG-Änderung einen alten Override gezielt zurückzusetzen. Danach unbedingt wieder auf false stellen, siehe Warnhinweis unten.
----------------	-------	--

Die fünf live-überschreibbaren Werte:

KVS-Key	überschreibt	Gültigkeitsbereich
zdmc_setpoint	CONFIG.setpoint	-50 ...50 (W)
zdmc_hysteresis	CONFIG.hysteresis	≥ 5 ...50 (W)
zdmc_dampingFactor	CONFIG.dampingFactor	$0,1 \leq x \leq 1$ (Werte außerhalb werden verworfen, letzter gültiger Wert bleibt bestehen)
zdmc_devN_dischargeAllowed	CONFIG.devices[N].dischargeAllowed	0 oder 1 – N = Array-Index des Geräts (siehe [devN] im Start-Banner, Kapitel 8.7). Ein Key pro Gerät, z. B. zdmc_dev0_dischargeAllowed, zdmc_dev1_dischargeAllowed, ...
zdmc_devN_reverse	CONFIG.devices[N].reverse	0 oder 1 – N = Array-Index des Geräts (siehe [devN] im Start-Banner, Kapitel 8.7). Ein Key pro Gerät, z. B. zdmc_dev0_reverse, zdmc_dev1_reverse, ...

⚠ Warnhinweis: Schreibvorgänge in den KVS-Speicher des Shelly

Der KVS ist **Flash-gestützt** – der interne Speicherchip des Shelly verträgt nur eine begrenzte Anzahl an Schreibzyklen. Das betrifft ausschließlich **Schreibvorgänge** (KVS.Set), **nicht** Lesevorgänge:

- **Lesen** (KVS.GetMany) passiert **jeden Zyklus** (alle paar Sekunden) – das belastet den Flash-Speicher **nicht**.
- **Geschrieben** wird vom Script selbst nur **einmal beim Start** (Seeding, siehe

Kapitel 4/8.6): normalerweise nur für tatsächlich fehlende Keys, bei kvsForceReseed: true dagegen bei **jedem** Neustart für **alle** Keys neu.

- Der eigentliche Verschleiß entsteht durch **externe** Schreibvorgänge,
 - z. B. aus Home Assistant. Gelegentliche, bewusste Bedienung (Klick auf einen Switch/Regler) ist unkritisch – problematisch wird es erst, wenn eine Automation **regelmäßig** (z. B. sekundlich per Polling oder bei jedem Sensor-Update) automatisch schreibt, statt nur bei tatsächlicher Nutzer-Interaktion.

Empfehlungen: - kvsForceReseed nur kurzzeitig auf true setzen, einmal neu starten lassen, danach **wieder auf false** zurückstellen – sonst schreibt **jeder** künftige Neustart (Redeploy, Stromausfall, Watchdog-Reset, ...) unnötig alle Keys neu. - Home-Assistant-Automatationen so bauen, dass KVS.Set nur bei tatsächlicher Nutzer-Interaktion feuert (siehe Beispiele unten, optimistische switch/number-Entities) – nicht zyklisch/gepollt. - Wird das Feature gar nicht genutzt: kvsEnabled: false setzen (siehe oben) – dann entfällt auch der lesende Aufruf pro Zyklus vollständig.

Unabhängig davon gilt: Die Werte im KVS bleiben auch bei gestopptem Script erhalten. Ein normaler Neustart (kvsForceReseed: false) überschreibt sie **nicht** – sie bilden weiterhin die Grundlage der Regelung.

Anwendungsbeispiel 1: Sollwert per curl setzen

Ganz ohne Home Assistant testbar, direkt im LAN gegen die Shelly-eigene RPC-API:

```
curl -X POST "http://<shelly-ip>/rpc/KVS.Set" \  
-H "Content-Type: application/json" \  
-d '{"key":"zdmc_setpoint","value":-50}'
```

Kontrolle, ob der Wert angekommen ist:

```
curl "http://<shelly-ip>/rpc/KVS.Get?key=zdmc_setpoint"
```

Spätestens im nächsten Regelzyklus wird der neue Sollwert übernommen (sichtbar auch im Shelly-Log).

Anwendungsbeispiel 1b: Ein Gerät gezielt vom Entladen ausschließen

Den Array-Index N des gewünschten Geräts vorher im Start-Banner ablesen ([devN]-Präfix vor dem Label, Kapitel 8.7). Beispiel: Gerät mit Index 1 (z. B. "SF800") soll nicht mehr entladen, aber weiter laden dürfen:

```
curl -X POST "http://<shelly-ip>/rpc/KVS.Set" \  
-H "Content-Type: application/json" \  
-d '{"key":"zdmc_dev1_dischargeAllowed","value":0}'
```

Wieder freigeben:

```
curl -X POST "http://<shelly-ip>/rpc/KVS.Set" \  
-H "Content-Type: application/json" \  
-d '{"key":"zdmc_dev1_dischargeAllowed","value":1}'
```

In Home Assistant eignet sich dafür ein rest_command + switch (Top-Level-Key template:) Duo – ohne separaten Helfer und ohne Automation, da der Switch selbst den Aufruf in turn_on/turn_off auslöst:

```
rest_command:  
  zdmc_set_dev1_discharge_allowed:  
    url: "http://<shelly-ip>/rpc/KVS.Set"  
    method: POST
```

```

content_type: "application/json"
payload: '{"key":"zdmc_dev1_dischargeAllowed","value":{{ value }}}'

template:
- switch:
  - name: "Zendure SF800 Entladen erlaubt"
    unique_id: zdmc_dev1_discharge_allowed
    turn_on:
      - action: rest_command.zdmc_set_dev1_discharge_allowed
        data:
          value: 1
    turn_off:
      - action: rest_command.zdmc_set_dev1_discharge_allowed
        data:
          value: 0

```

Den Switch als normale Schalter-/Button-Karte ins Dashboard legen – ein Klick löst sofort den KVS-Aufruf aus, keine zusätzliche Automation nötig. Für weitere Geräte das Duo duplizieren und dev1/SF800 durch den jeweiligen Index/Namen ersetzen (Key, rest_command-Name und Switch-Entity konsistent halten). **Hinweis:** Da bewusst kein state-Template angegeben ist, läuft der Switch optimistisch (Standardverhalten bei fehlendem state) – er zeigt an, was zuletzt geklickt wurde, ohne Rückmeldung, ob der Shelly den Wert tatsächlich übernommen hat.

Anwendungsbeispiel 1c: Laden vom Netz (reverse) für ein Gerät umschalten

Funktioniert analog zu 1b, nur mit zdmc_devN_reverse statt zdmc_devN_dischargeAllowed. Beispiel: Gerät mit Index 1 (z. B. "SF800") soll zusätzlich gezielt aus dem Netz laden dürfen:

```

curl -X POST "http://<shelly-ip>/rpc/KVS.Set" \
-H "Content-Type: application/json" \
-d '{"key":"zdmc_dev1_reverse","value":1}'

```

Wieder sperren:

```

curl -X POST "http://<shelly-ip>/rpc/KVS.Set" \
-H "Content-Type: application/json" \
-d '{"key":"zdmc_dev1_reverse","value":0}'

```

In Home Assistant dasselbe rest_command + switch-Duo wie in 1b, nur mit eigenem Key/Namen:

```

rest_command:
  zdmc_set_dev1_reverse:
    url: "http://<shelly-ip>/rpc/KVS.Set"
    method: POST
    content_type: "application/json"
    payload: '{"key":"zdmc_dev1_reverse","value":{{ value }}}'

template:
- switch:
  - name: "Zendure SF800 Laden vom Netz erlaubt"
    unique_id: zdmc_dev1_reverse
    turn_on:
      - action: rest_command.zdmc_set_dev1_reverse
        data:

```

```

        value: 1
    turn_off:
      - action: rest_command.zdmc_set_dev1_reverse
        data:
          value: 0

```

Auch hier gilt: optimistischer Switch (kein state-Template), Hinweise wie in Beispiel 1b.

Anwendungsbeispiel 2: Home Assistant

Wie in Beispiel 1b: kein separater Helfer und keine Automation nötig. Jede der drei globalen Regelgrößen (setpoint, hysteresis, dampingFactor) bekommt eine number-Template-Entity, die den passenden rest_command direkt in ihrer eigenen set_value-Aktion aufruft. Die beiden Geräte-Schalter dischargeAllowed und reverse sind boolesch und daher über switch-Entities abgedeckt (Beispiel 1b/1c).

- 1) rest_command anlegen – ein Befehl pro Wert (in configuration.yaml, Top-Level-Key rest_command:):

```

rest_command:
  zdmc_set_setpoint:
    url: "http://<shelly-ip>/rpc/KVS.Set"
    method: POST
    content_type: "application/json"
    payload: '{"key":"zdmc_setpoint","value":{{ value }}}'

  zdmc_set_hysteresis:
    url: "http://<shelly-ip>/rpc/KVS.Set"
    method: POST
    content_type: "application/json"
    payload: '{"key":"zdmc_hysteresis","value":{{ value }}}'

  zdmc_set_damping_factor:
    url: "http://<shelly-ip>/rpc/KVS.Set"
    method: POST
    content_type: "application/json"
    payload: '{"key":"zdmc_dampingFactor","value":{{ value }}}'

```

value ist dabei kein Zufallsname – rest_command reicht jedes zusätzliche Datenfeld aus dem Aktionsaufruf 1:1 als Template-Variable in den payload durch (siehe Schritt 2, data: value: ...).

- 2) number-Template-Entities anlegen (Top-Level-Key template:), je eine pro Regelgröße, mit sinnvollen Slider-Grenzen (die eigentliche Absicherung bleibt aber die KVS-Validierung im Script selbst, siehe Kapitel 4):

```

template:
  - number:
      - name: "Zendure Setpoint"
        unique_id: zdmc_setpoint
        min: -1000
        max: 1000
        step: 10
        unit_of_measurement: "W"
        icon: mdi:transmission-tower

```

```

set_value:
  - action: rest_command.zdmc_set_setpoint
    data:
      value: "{{ value }}"

- name: "Zendure Hysteresis"
  unique_id: zdmc_hysteresis
  min: 0
  max: 100
  step: 10
  unit_of_measurement: "W"
  icon: mdi:sine-wave
  set_value:
    - action: rest_command.zdmc_set_hysteresis
      data:
        value: "{{ value }}"

- name: "Zendure Damping Factor"
  unique_id: zdmc_damping_factor
  min: 0.1
  max: 1.0
  step: 0.1
  icon: mdi:speedometer-slow
  set_value:
    - action: rest_command.zdmc_set_damping_factor
      data:
        value: "{{ value }}"

```

Alle drei Entities landen als normale Zahlen-/Schieberegler-Karten im Dashboard, ganz ohne input_number-Helfer und ohne automation:-Block. **Hinweis:** Da bewusst kein state-Template angegeben ist, laufen die Entities im optimistischen Modus (Standardverhalten bei fehlendem state) – der angezeigte Wert ist der zuletzt eingestellte, ohne Rückmeldung, ob der Shelly ihn tatsächlich übernommen hat, und ohne Vorbelegung (initial) direkt nach einem HA-Neustart, bis einmal am Regler gezogen wurde. Der zuletzt per KVS gesetzte Wert im Shelly bleibt davon unberührt.

KVS Store im Shelly

(Screenshot des KVS-Stores in der Shelly-Web-UI: siehe Geräte-Oberfläche unter „Scripts“ → „KVS“.)